Étude particulière du dataset "users-score-2023.csv" :

```
import pandas as pd
import gc
from surprise import Dataset, Reader, accuracy, SVD, SVDpp
from surprise.model_selection import train_test_split, GridSearchCV
from surprise.prediction_algorithms.knns import KNNBasic,
KNNWithMeans, KNNWithZScore, KNNBaseline
from surprise.prediction_algorithms.slope_one import SlopeOne
from surprise.prediction_algorithms.co_clustering import CoClustering
from sklearn.neighbors import NearestNeighbors

# Charger le dataset
user_score = pd.read_csv("./users-score-2023.csv")

# Aperçu du dataset
print(user_score.head())
print(user_score.info())
```

```
   user_id Username  anime_id             Anime Title  rating
0        1    Xinil        21               One Piece       9
1        1    Xinil        48             .hack//Sign       7
2        1    Xinil       320                  A Kite       5
3        1    Xinil        49        Aa! Megami-sama!       8
4        1    Xinil       304  Aa! Megami-sama! Movie       8
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 24325191 entries, 0 to 24325190
Data columns (total 5 columns):
 #   Column       Dtype
---  ------       -----
 0   user_id      int64
 1   Username     object
 2   anime_id     int64
 3   Anime Title  object
 4   rating       int64
dtypes: int64(3), object(2)
memory usage: 927.9+ MB
None
```

Restreindre le dataset

```
# Fixer les seuils
min_user_ratings = 50  # Par exemple, les utilisateurs doivent avoir
au moins noté 50 animes
min_anime_ratings = 10_000  # Par exemple, les animes doivent avoir au
moins 10 000 notes

# Compter le nombre de notes par utilisateur et par anime
user_rating_counts = user_score.groupby('user_id').size()
anime_rating_counts = user_score.groupby('anime_id').size()
```

```
# Filtrer les utilisateurs avec au moins 10 notes et les animes avec au moins 100,000 notes
filtered_users = user_rating_counts[user_rating_counts >= min_user_ratings].index
filtered_animes = anime_rating_counts[anime_rating_counts >= min_anime_ratings].index

# Restreindre le dataset aux utilisateurs et animes filtrés
filtered_scores = user_score[user_score['user_id'].isin(filtered_users) & user_score['anime_id'].isin(filtered_animes)]
```

```
user_rating_counts
```

```
user_id
1          262
4          284
9           65
20         105
23         282
          ...
1291057    240
1291079     94
1291085     59
1291087    210
1291097      3
Length: 270033, dtype: int64
```

```
filtered_users
```

```
Index([      1,       4,       9,      20,      23,      37,      47,
48,
            53,      66,
       ...
       1291015, 1291017, 1291021, 1291029, 1291033, 1291049, 1291057,
1291079,
       1291085, 1291087],
      dtype='int64', name='user_id', length=117568)
```

```
filtered_scores
```

```
          user_id   Username  anime_id  \
0               1      Xinil        21
1               1      Xinil        48
6               1      Xinil        53
7               1      Xinil        47
8               1      Xinil       591
...           ...        ...       ...
24325178  1291087  Oblongata     14813
24325179  1291087  Oblongata      8861
```

```
24325183  1291087  Oblongata     10578
24325184  1291087  Oblongata     12549
24325187  1291087  Oblongata       174

                                                Anime Title  rating
0                                                 One Piece       9
1                                                .hack//Sign       7
6                                           Ai Yori Aoshi       7
7                                                   Akira       5
8                                         Amaenaide yo!!       6
...                                                  ...     ...
24325178  Yahari Ore no Seishun Love Comedy wa Machigatt...       8
24325179  Yosuga no Sora: In Solitude, Where We Are Leas...       3
24325183                                                C³       6
24325184                     Dakara Boku wa, H ga Dekinai.       4
24325187                                      Tenjou Tenge       6

[11219809 rows x 5 columns]
```

```python
import gc

# Supprimer la variable
del user_score

# Forcer le garbage collector à libérer la mémoire
gc.collect()

# filtered_scores devient user_score on prend juste un échantillon des
1000 premiers animés
filtered_scores = filtered_scores.head(1000)

# Vérifier la taille du dataset après filtrage
print(f"Nombre d'utilisateurs filtrés : {len(filtered_users)}")
print(f"Nombre d'animes filtrés : {len(filtered_animes)}")
print(f"Nombre de notes après filtrage : {len(filtered_scores)}")
```

```
Nombre d'utilisateurs filtrés : 117568
Nombre d'animes filtrés : 598
Nombre de notes après filtrage : 1000
```

Collaborative Filtering

```python
# Créer le dataset pour Surprise
reader = Reader(rating_scale=(1, 10))  # Assuming rating scale is from
1 to 10
data = Dataset.load_from_df(filtered_scores[['user_id', 'anime_id',
'rating']], reader)

data
```

```
<surprise.dataset.DatasetAutoFolds at 0x1d82a345c10>
```

```
train, test = train_test_split(data, test_size=0.2)

print(type(train))
print(type(test))
```

```
<class 'surprise.trainset.Trainset'>
<class 'list'>
```

```
cpt=0
for r in train.all_ratings():
  print(r)
  cpt+=1
  if cpt > 20:
    break
```

```
(0, 0, 8.0)
(0, 1, 7.0)
(0, 10, 6.0)
(0, 24, 6.0)
(0, 31, 5.0)
(0, 39, 7.0)
(0, 41, 8.0)
(0, 48, 8.0)
(0, 50, 9.0)
(0, 53, 7.0)
(0, 59, 8.0)
(0, 60, 8.0)
(0, 63, 2.0)
(0, 5, 7.0)
(0, 64, 7.0)
(0, 69, 6.0)
(0, 29, 7.0)
(0, 75, 9.0)
(0, 79, 7.0)
(0, 23, 7.0)
(0, 94, 8.0)
```

Construction du jeu antitest

```
full_train = data.build_full_trainset()
antitest = full_train.build_anti_testset()

print(antitest)
```

```
[(1, 298, 7.541), (1, 5525, 7.541), (1, 6682, 7.541), (1, 11759,
7.541), (1, 101, 7.541), (1, 102, 7.541), (1, 22199, 7.541), (1, 4744,
7.541), (1, 25013, 7.541), (1, 22729, 7.541), (1, 27655, 7.541), (1,
6547, 7.541), (1, 9989, 7.541), (1, 11433, 7.541), (1, 11111, 7.541),
(1, 24833, 7.541), (1, 21995, 7.541), (1, 9919, 7.541), (1, 12967,
7.541), (1, 477, 7.541), (1, 9736, 7.541), (1, 5342, 7.541), (1, 2251,
7.541), (1, 3901, 7.541), (1, 6347, 7.541), (1, 7674, 7.541), (1,
```

10030, 7.541), (1, 2986, 7.541), (1, 22789, 7.541), (1, 10396, 7.541), (1, 13535, 7.541), (1, 20787, 7.541), (1, 7059, 7.541), (1, 10490, 7.541), (1, 31043, 7.541), (1, 31964, 7.541), (1, 33486, 7.541), (1, 36456, 7.541), (1, 10719, 7.541), (1, 14967, 7.541), (1, 6772, 7.541), (1, 8514, 7.541), (1, 14345, 7.541), (1, 31478, 7.541), (1, 1536, 7.541), (1, 10163, 7.541), (1, 12293, 7.541), (1, 5356, 7.541), (1, 10578, 7.541), (1, 28999, 7.541), (1, 10800, 7.541), (1, 14397, 7.541), (1, 4186, 7.541), (1, 14741, 7.541), (1, 18671, 7.541), (1, 2167, 7.541), (1, 11703, 7.541), (1, 12549, 7.541), (1, 16592, 7.541), (1, 35849, 7.541), (1, 15583, 7.541), (1, 28223, 7.541), (1, 8086, 7.541), (1, 9330, 7.541), (1, 28121, 7.541), (1, 2924, 7.541), (1, 1250, 7.541), (1, 2759, 7.541), (1, 3784, 7.541), (1, 3785, 7.541), (1, 8536, 7.541), (1, 18507, 7.541), (1, 9367, 7.541), (1, 5114, 7.541), (1, 908, 7.541), (1, 430, 7.541), (1, 123, 7.541), (1, 4725, 7.541), (1, 25183, 7.541), (1, 17895, 7.541), (1, 8425, 7.541), (1, 10793, 7.541), (1, 134, 7.541), (1, 13161, 7.541), (1, 31859, 7.541), (1, 20583, 7.541), (1, 32935, 7.541), (1, 28891, 7.541), (1, 587, 7.541), (1, 15225, 7.541), (1, 8630, 7.541), (1, 8074, 7.541), (1, 9515, 7.541), (1, 934, 7.541), (1, 1889, 7.541), (1, 9041, 7.541), (1, 3731, 7.541), (1, 228, 7.541), (1, 3092, 7.541), (1, 4814, 7.541), (1, 14467, 7.541), (1, 5680, 7.541), (1, 6862, 7.541), (1, 7054, 7.541), (1, 813, 7.541), (1, 467, 7.541), (1, 379, 7.541), (1, 790, 7.541), (1, 10087, 7.541), (1, 11741, 7.541), (1, 395, 7.541), (1, 578, 7.541), (1, 14719, 7.541), (1, 4282, 7.541), (1, 572, 7.541), (1, 34599, 7.541), (1, 512, 7.541), (1, 3655, 7.541), (1, 4437, 7.541), (1, 594, 7.541), (1, 4477, 7.541), (1, 29803, 7.541), (1, 528, 7.541), (1, 166, 7.541), (1, 9253, 7.541), (1, 16524, 7.541), (1, 11757, 7.541), (1, 6702, 7.541), (1, 1482, 7.541), (1, 28171, 7.541), (1, 1292, 7.541), (1, 7647, 7.541), (1, 9513, 7.541), (1, 1498, 7.541), (1, 4901, 7.541), (1, 11285, 7.541), (1, 1686, 7.541), (1, 2889, 7.541), (1, 762, 7.541), (1, 834, 7.541), (1, 11633, 7.541), (1, 405, 7.541), (1, 11241, 7.541), (1, 4981, 7.541), (1, 8915, 7.541), (1, 4182, 7.541), (1, 7338, 7.541), (1, 6573, 7.541), (1, 6880, 7.541), (1, 10155, 7.541), (1, 223, 7.541), (1, 225, 7.541), (1, 6033, 7.541), (1, 22297, 7.541), (1, 28701, 7.541), (1, 4192, 7.541), (1, 11617, 7.541), (1, 8277, 7.541), (1, 7088, 7.541), (1, 468, 7.541), (1, 18679, 7.541), (1, 22535, 7.541), (1, 801, 7.541), (1, 237, 7.541), (1, 4898, 7.541), (1, 6707, 7.541), (1, 18153, 7.541), (1, 17265, 7.541), (1, 23321, 7.541), (1, 18115, 7.541), (1, 14513, 7.541), (1, 20785, 7.541), (1, 23755, 7.541), (1, 442, 7.541), (1, 19815, 7.541), (1, 20507, 7.541), (1, 13601, 7.541), (1, 23281, 7.541), (1, 2993, 7.541), (1, 4214, 7.541), (1, 64, 7.541), (1, 4063, 7.541), (1, 5355, 7.541), (1, 154, 7.541), (1, 3588, 7.541), (1, 2418, 7.541), (1, 21881, 7.541), (1, 9941, 7.541), (1, 6213, 7.541), (1, 4654, 7.541), (1, 8937, 7.541), (1, 22319, 7.541), (1, 27899, 7.541), (1, 3167, 7.541), (1, 2787, 7.541), (1, 530, 7.541), (1, 740, 7.541), (1, 14713, 7.541), (1, 3503, 7.541), (1, 6045, 7.541), (1, 9656, 7.541), (1, 355, 7.541), (1, 4722, 7.541), (1, 534, 7.541), (1, 3470, 7.541), (1, 513, 7.541), (1, 14227, 7.541), (1, 3457, 7.541), (1, 4752, 7.541), (1,

4181, 7.541), (1, 10620, 7.541), (1, 2923, 7.541), (1, 713, 7.541), (1, 357, 7.541), (1, 6351, 7.541), (1, 4059, 7.541), (1, 376, 7.541), (1, 3958, 7.541), (1, 1004, 7.541), (1, 1887, 7.541), (1, 4472, 7.541), (1, 2963, 7.541), (1, 527, 7.541), (1, 1117, 7.541), (1, 1118, 7.541), (1, 846, 7.541), (1, 1965, 7.541), (1, 4382, 7.541), (1, 1195, 7.541), (1, 232, 7.541), (1, 94, 7.541), (1, 79, 7.541), (1, 177, 7.541), (1, 1840, 7.541), (1, 481, 7.541), (1, 9062, 7.541), (1, 6746, 7.541), (1, 9314, 7.541), (1, 6594, 7.541), (1, 8129, 7.541), (1, 9756, 7.541), (1, 7769, 7.541), (1, 10067, 7.541), (1, 5081, 7.541), (1, 4835, 7.541), (1, 4177, 7.541), (1, 2164, 7.541), (1, 5177, 7.541), (1, 1726, 7.541), (1, 552, 7.541), (1, 874, 7.541), (1, 3230, 7.541), (1, 408, 7.541), (1, 387, 7.541), (1, 6895, 7.541), (1, 452, 7.541), (1, 450, 7.541), (1, 451, 7.541), (1, 7791, 7.541), (1, 2593, 7.541), (1, 3782, 7.541), (1, 3783, 7.541), (1, 4280, 7.541), (1, 5204, 7.541), (1, 5205, 7.541), (1, 1691, 7.541), (1, 6205, 7.541), (1, 1579, 7.541), (1, 3572, 7.541), (1, 98, 7.541), (1, 3342, 7.541), (1, 4081, 7.541), (1, 6512, 7.541), (1, 4155, 7.541), (1, 5341, 7.541), (1, 6007, 7.541), (1, 8769, 7.541), (1, 437, 7.541), (1, 65, 7.541), (1, 339, 7.541), (1, 5681, 7.541), (1, 7311, 7.541), (1, 6211, 7.541), (1, 4224, 7.541), (1, 27, 7.541), (1, 2129, 7.541), (1, 969, 7.541), (1, 6956, 7.541), (1, 861, 7.541), (1, 3712, 7.541), (1, 5300, 7.541), (1, 4975, 7.541), (1, 6747, 7.541), (1, 2581, 7.541), (1, 5530, 7.541), (1, 5682, 7.541), (1, 6500, 7.541), (1, 3091, 7.541), (1, 67, 7.541), (1, 149, 7.541), (1, 5909, 7.541), (1, 371, 7.541), (1, 372, 7.541), (1, 1313, 7.541), (1, 2961, 7.541), (1, 1132, 7.541), (4, 21, 7.541), (4, 48, 7.541), (4, 53, 7.541), (4, 591, 7.541), (4, 54, 7.541), (4, 57, 7.541), (4, 1519, 7.541), (4, 59, 7.541), (4, 1818, 7.541), (4, 226, 7.541), (4, 356, 7.541), (4, 1361, 7.541), (4, 227, 7.541), (4, 73, 7.541), (4, 1015, 7.541), (4, 72, 7.541), (4, 2890, 7.541), (4, 239, 7.541), (4, 384, 7.541), (4, 2508, 7.541), (4, 241, 7.541), (4, 268, 7.541), (4, 267, 7.541), (4, 16, 7.541), (4, 263, 7.541), (4, 5258, 7.541), (4, 5630, 7.541), (4, 256, 7.541), (4, 431, 7.541), (4, 11061, 7.541), (4, 257, 7.541), (4, 185, 7.541), (4, 249, 7.541), (4, 33, 7.541), (4, 93, 7.541), (4, 147, 7.541), (4, 32281, 7.541), (4, 486, 7.541), (4, 412, 7.541), (4, 43, 7.541), (4, 433, 7.541), (4, 416, 7.541), (4, 97, 7.541), (4, 189, 7.541), (4, 190, 7.541), (4, 191, 7.541), (4, 192, 7.541), (4, 2034, 7.541), (4, 330, 7.541), (4, 585, 7.541), (4, 164, 7.541), (4, 19, 7.541), (4, 457, 7.541), (4, 877, 7.541), (4, 20, 7.541), (4, 936, 7.541), (4, 1735, 7.541), (4, 597, 7.541), (4, 30, 7.541), (4, 31, 7.541), (4, 32, 7.541), (4, 1210, 7.541), (4, 1698, 7.541), (4, 30276, 7.541), (4, 195, 7.541), (4, 196, 7.541), (4, 12355, 7.541), (4, 2966, 7.541), (4, 853, 7.541), (4, 1943, 7.541), (4, 322, 7.541), (4, 210, 7.541), (4, 710, 7.541), (4, 45, 7.541), (4, 401, 7.541), (4, 44, 7.541), (4, 205, 7.541), (4, 2476, 7.541), (4, 24, 7.541), (4, 400, 7.541), (4, 1827, 7.541), (4, 199, 7.541), (4, 2104, 7.541), (4, 1559, 7.541), (4, 90, 7.541), (4, 16498, 7.541), (4, 25777, 7.541), (4, 169, 7.541), (4, 849, 7.541), (4, 2001, 7.541), (4, 174, 7.541), (4, 182, 7.541), (4, 2236, 7.541), (4, 523, 7.541), (4, 6, 7.541), (4, 10162, 7.541), (4,

7, 7.541), (4, 202, 7.541), (4, 1562, 7.541), (4, 392, 7.541), (4, 269, 7.541), (4, 15, 7.541), (4, 1699, 7.541), (4, 2605, 7.541), (4, 22, 7.541), (4, 857, 7.541), (4, 66, 7.541), (4, 1535, 7.541), (4, 235, 7.541), (4, 3002, 7.541), (4, 2026, 7.541), (4, 777, 7.541), (4, 1530, 7.541), (4, 145, 7.541), (4, 1604, 7.541), (4, 157, 7.541), (4, 323, 7.541), (4, 813, 7.541), (4, 467, 7.541), (4, 379, 7.541), (4, 790, 7.541), (4, 10087, 7.541), (4, 11741, 7.541), (4, 395, 7.541), (4, 578, 7.541), (4, 14719, 7.541), (4, 4282, 7.541), (4, 572, 7.541), (4, 34599, 7.541), (4, 512, 7.541), (4, 3655, 7.541), (4, 4437, 7.541), (4, 594, 7.541), (4, 4477, 7.541), (4, 29803, 7.541), (4, 528, 7.541), (4, 166, 7.541), (4, 9253, 7.541), (4, 16524, 7.541), (4, 11757, 7.541), (4, 6702, 7.541), (4, 1482, 7.541), (4, 28171, 7.541), (4, 1292, 7.541), (4, 7647, 7.541), (4, 9513, 7.541), (4, 1498, 7.541), (4, 4901, 7.541), (4, 11285, 7.541), (4, 1686, 7.541), (4, 2889, 7.541), (4, 762, 7.541), (4, 834, 7.541), (4, 11633, 7.541), (4, 405, 7.541), (4, 11241, 7.541), (4, 4981, 7.541), (4, 8915, 7.541), (4, 4182, 7.541), (4, 7338, 7.541), (4, 6573, 7.541), (4, 6880, 7.541), (4, 10155, 7.541), (4, 223, 7.541), (4, 225, 7.541), (4, 6033, 7.541), (4, 22297, 7.541), (4, 28701, 7.541), (4, 4192, 7.541), (4, 11617, 7.541), (4, 8277, 7.541), (4, 7088, 7.541), (4, 468, 7.541), (4, 18679, 7.541), (4, 22535, 7.541), (4, 801, 7.541), (4, 237, 7.541), (4, 4898, 7.541), (4, 6707, 7.541), (4, 18153, 7.541), (4, 17265, 7.541), (4, 23321, 7.541), (4, 18115, 7.541), (4, 14513, 7.541), (4, 20785, 7.541), (4, 23755, 7.541), (4, 442, 7.541), (4, 19815, 7.541), (4, 20507, 7.541), (4, 13601, 7.541), (4, 23281, 7.541), (4, 2993, 7.541), (4, 4214, 7.541), (4, 64, 7.541), (4, 4063, 7.541), (4, 5355, 7.541), (4, 154, 7.541), (4, 3588, 7.541), (4, 2418, 7.541), (4, 21881, 7.541), (4, 9941, 7.541), (4, 6213, 7.541), (4, 4654, 7.541), (4, 8937, 7.541), (4, 22319, 7.541), (4, 27899, 7.541), (4, 3167, 7.541), (4, 2787, 7.541), (4, 530, 7.541), (4, 740, 7.541), (4, 14713, 7.541), (4, 3503, 7.541), (4, 6045, 7.541), (4, 9656, 7.541), (4, 355, 7.541), (4, 4722, 7.541), (4, 534, 7.541), (4, 3470, 7.541), (4, 513, 7.541), (4, 14227, 7.541), (4, 3457, 7.541), (4, 4752, 7.541), (4, 4181, 7.541), (4, 10620, 7.541), (4, 2923, 7.541), (4, 713, 7.541), (4, 357, 7.541), (4, 6351, 7.541), (4, 4059, 7.541), (4, 376, 7.541), (4, 3958, 7.541), (4, 1004, 7.541), (4, 1887, 7.541), (4, 4472, 7.541), (4, 2963, 7.541), (4, 527, 7.541), (4, 1117, 7.541), (4, 1118, 7.541), (4, 846, 7.541), (4, 1965, 7.541), (4, 4382, 7.541), (4, 1195, 7.541), (4, 232, 7.541), (4, 94, 7.541), (4, 79, 7.541), (4, 177, 7.541), (4, 1840, 7.541), (4, 481, 7.541), (4, 9062, 7.541), (4, 6746, 7.541), (4, 9314, 7.541), (4, 6594, 7.541), (4, 8129, 7.541), (4, 9756, 7.541), (4, 7769, 7.541), (4, 10067, 7.541), (4, 5081, 7.541), (4, 4835, 7.541), (4, 4177, 7.541), (4, 2164, 7.541), (4, 5177, 7.541), (4, 1726, 7.541), (4, 552, 7.541), (4, 874, 7.541), (4, 3230, 7.541), (4, 408, 7.541), (4, 387, 7.541), (4, 6895, 7.541), (4, 452, 7.541), (4, 450, 7.541), (4, 451, 7.541), (4, 7791, 7.541), (4, 2593, 7.541), (4, 3782, 7.541), (4, 3783, 7.541), (4, 4280, 7.541), (4, 5204, 7.541), (4, 5205, 7.541), (4, 1691, 7.541), (4, 6205, 7.541), (4, 1579, 7.541), (4, 3572, 7.541), (4, 98, 7.541), (4, 3342,

7.541), (4, 4081, 7.541), (4, 6512, 7.541), (4, 4155, 7.541), (4, 5341, 7.541), (4, 6007, 7.541), (4, 8769, 7.541), (4, 437, 7.541), (4, 65, 7.541), (4, 339, 7.541), (4, 5681, 7.541), (4, 7311, 7.541), (4, 6211, 7.541), (4, 4224, 7.541), (4, 27, 7.541), (4, 2129, 7.541), (4, 969, 7.541), (4, 6956, 7.541), (4, 861, 7.541), (4, 3712, 7.541), (4, 5300, 7.541), (4, 4975, 7.541), (4, 6747, 7.541), (4, 2581, 7.541), (4, 5530, 7.541), (4, 5682, 7.541), (4, 6500, 7.541), (4, 3091, 7.541), (4, 67, 7.541), (4, 149, 7.541), (4, 5909, 7.541), (4, 371, 7.541), (4, 372, 7.541), (4, 1313, 7.541), (4, 2961, 7.541), (4, 1132, 7.541), (9, 21, 7.541), (9, 48, 7.541), (9, 47, 7.541), (9, 591, 7.541), (9, 54, 7.541), (9, 57, 7.541), (9, 68, 7.541), (9, 889, 7.541), (9, 1519, 7.541), (9, 1222, 7.541), (9, 1689, 7.541), (9, 1818, 7.541), (9, 1575, 7.541), (9, 1, 7.541), (9, 5, 7.541), (9, 2025, 7.541), (9, 356, 7.541), (9, 1361, 7.541), (9, 227, 7.541), (9, 120, 7.541), (9, 71, 7.541), (9, 1015, 7.541), (9, 122, 7.541), (9, 2890, 7.541), (9, 239, 7.541), (9, 384, 7.541), (9, 240, 7.541), (9, 2508, 7.541), (9, 132, 7.541), (9, 1571, 7.541), (9, 268, 7.541), (9, 245, 7.541), (9, 133, 7.541), (9, 267, 7.541), (9, 16, 7.541), (9, 263, 7.541), (9, 5258, 7.541), (9, 15809, 7.541), (9, 5630, 7.541), (9, 135, 7.541), (9, 256, 7.541), (9, 431, 7.541), (9, 136, 7.541), (9, 11061, 7.541), (9, 138, 7.541), (9, 139, 7.541), (9, 137, 7.541), (9, 248, 7.541), (9, 185, 7.541), (9, 249, 7.541), (9, 33, 7.541), (9, 32281, 7.541), (9, 486, 7.541), (9, 412, 7.541), (9, 43, 7.541), (9, 433, 7.541), (9, 416, 7.541), (9, 97, 7.541), (9, 190, 7.541), (9, 191, 7.541), (9, 192, 7.541), (9, 2034, 7.541), (9, 585, 7.541), (9, 164, 7.541), (9, 19, 7.541), (9, 457, 7.541), (9, 877, 7.541), (9, 936, 7.541), (9, 1735, 7.541), (9, 597, 7.541), (9, 31, 7.541), (9, 32, 7.541), (9, 1210, 7.541), (9, 1698, 7.541), (9, 30276, 7.541), (9, 195, 7.541), (9, 12355, 7.541), (9, 2966, 7.541), (9, 853, 7.541), (9, 1943, 7.541), (9, 322, 7.541), (9, 210, 7.541), (9, 710, 7.541), (9, 401, 7.541), (9, 44, 7.541), (9, 2476, 7.541), (9, 400, 7.541), (9, 1827, 7.541), (9, 199, 7.541), (9, 2104, 7.541), (9, 1559, 7.541), (9, 90, 7.541), (9, 16498, 7.541), (9, 25777, 7.541), (9, 169, 7.541), (9, 849, 7.541), (9, 2001, 7.541), (9, 182, 7.541), (9, 2236, 7.541), (9, 523, 7.541), (9, 10162, 7.541), (9, 7, 7.541), (9, 1562, 7.541), (9, 15, 7.541), (9, 1699, 7.541), (9, 2605, 7.541), (9, 857, 7.541), (9, 66, 7.541), (9, 150, 7.541), (9, 2904, 7.541), (9, 1535, 7.541), (9, 235, 7.541), (9, 3002, 7.541), (9, 2026, 7.541), (9, 1530, 7.541), (9, 145, 7.541), (9, 1604, 7.541), (9, 157, 7.541), (9, 323, 7.541), (9, 298, 7.541), (9, 5525, 7.541), (9, 6682, 7.541), (9, 11759, 7.541), (9, 102, 7.541), (9, 22199, 7.541), (9, 4744, 7.541), (9, 25013, 7.541), (9, 22729, 7.541), (9, 27655, 7.541), (9, 6547, 7.541), (9, 9989, 7.541), (9, 11433, 7.541), (9, 11111, 7.541), (9, 24833, 7.541), (9, 21995, 7.541), (9, 9919, 7.541), (9, 12967, 7.541), (9, 477, 7.541), (9, 9736, 7.541), (9, 5342, 7.541), (9, 2251, 7.541), (9, 3901, 7.541), (9, 6347, 7.541), (9, 7674, 7.541), (9, 10030, 7.541), (9, 2986, 7.541), (9, 22789, 7.541), (9, 10396, 7.541), (9, 13535, 7.541), (9, 20787, 7.541), (9, 7059, 7.541), (9, 10490, 7.541), (9, 31043, 7.541), (9, 31964, 7.541), (9, 33486, 7.541), (9, 36456,

7.541), (9, 10719, 7.541), (9, 14967, 7.541), (9, 6772, 7.541), (9, 8514, 7.541), (9, 14345, 7.541), (9, 31478, 7.541), (9, 1536, 7.541), (9, 10163, 7.541), (9, 12293, 7.541), (9, 5356, 7.541), (9, 10578, 7.541), (9, 28999, 7.541), (9, 10800, 7.541), (9, 14397, 7.541), (9, 4186, 7.541), (9, 14741, 7.541), (9, 18671, 7.541), (9, 2167, 7.541), (9, 11703, 7.541), (9, 12549, 7.541), (9, 16592, 7.541), (9, 35849, 7.541), (9, 15583, 7.541), (9, 28223, 7.541), (9, 8086, 7.541), (9, 9330, 7.541), (9, 28121, 7.541), (9, 2924, 7.541), (9, 1250, 7.541), (9, 2759, 7.541), (9, 3784, 7.541), (9, 3785, 7.541), (9, 8536, 7.541), (9, 18507, 7.541), (9, 9367, 7.541), (9, 5114, 7.541), (9, 908, 7.541), (9, 123, 7.541), (9, 4725, 7.541), (9, 25183, 7.541), (9, 17895, 7.541), (9, 8425, 7.541), (9, 10793, 7.541), (9, 13161, 7.541), (9, 31859, 7.541), (9, 20583, 7.541), (9, 32935, 7.541), (9, 28891, 7.541), (9, 587, 7.541), (9, 15225, 7.541), (9, 8630, 7.541), (9, 8074, 7.541), (9, 9515, 7.541), (9, 934, 7.541), (9, 1889, 7.541), (9, 9041, 7.541), (9, 3731, 7.541), (9, 228, 7.541), (9, 3092, 7.541), (9, 4814, 7.541), (9, 14467, 7.541), (9, 5680, 7.541), (9, 6862, 7.541), (9, 7054, 7.541), (9, 790, 7.541), (9, 10087, 7.541), (9, 11741, 7.541), (9, 395, 7.541), (9, 578, 7.541), (9, 14719, 7.541), (9, 4282, 7.541), (9, 572, 7.541), (9, 34599, 7.541), (9, 512, 7.541), (9, 3655, 7.541), (9, 4437, 7.541), (9, 594, 7.541), (9, 4477, 7.541), (9, 29803, 7.541), (9, 528, 7.541), (9, 166, 7.541), (9, 9253, 7.541), (9, 16524, 7.541), (9, 11757, 7.541), (9, 6702, 7.541), (9, 1482, 7.541), (9, 28171, 7.541), (9, 1292, 7.541), (9, 7647, 7.541), (9, 9513, 7.541), (9, 1498, 7.541), (9, 4901, 7.541), (9, 11285, 7.541), (9, 1686, 7.541), (9, 2889, 7.541), (9, 762, 7.541), (9, 834, 7.541), (9, 11633, 7.541), (9, 405, 7.541), (9, 11241, 7.541), (9, 4981, 7.541), (9, 8915, 7.541), (9, 4182, 7.541), (9, 7338, 7.541), (9, 6573, 7.541), (9, 6880, 7.541), (9, 10155, 7.541), (9, 223, 7.541), (9, 225, 7.541), (9, 6033, 7.541), (9, 22297, 7.541), (9, 28701, 7.541), (9, 4192, 7.541), (9, 11617, 7.541), (9, 8277, 7.541), (9, 7088, 7.541), (9, 468, 7.541), (9, 18679, 7.541), (9, 22535, 7.541), (9, 801, 7.541), (9, 237, 7.541), (9, 4898, 7.541), (9, 6707, 7.541), (9, 18153, 7.541), (9, 17265, 7.541), (9, 23321, 7.541), (9, 18115, 7.541), (9, 14513, 7.541), (9, 20785, 7.541), (9, 23755, 7.541), (9, 442, 7.541), (9, 19815, 7.541), (9, 20507, 7.541), (9, 13601, 7.541), (9, 23281, 7.541), (9, 2993, 7.541), (9, 4214, 7.541), (9, 64, 7.541), (9, 4063, 7.541), (9, 5355, 7.541), (9, 154, 7.541), (9, 3588, 7.541), (9, 2418, 7.541), (9, 21881, 7.541), (9, 9941, 7.541), (9, 6213, 7.541), (9, 4654, 7.541), (9, 8937, 7.541), (9, 22319, 7.541), (9, 27899, 7.541), (9, 3167, 7.541), (9, 2787, 7.541), (9, 530, 7.541), (9, 740, 7.541), (9, 14713, 7.541), (9, 3503, 7.541), (9, 6045, 7.541), (9, 9656, 7.541), (9, 355, 7.541), (9, 4722, 7.541), (9, 534, 7.541), (9, 3470, 7.541), (9, 513, 7.541), (9, 14227, 7.541), (9, 3457, 7.541), (9, 4752, 7.541), (9, 4181, 7.541), (9, 10620, 7.541), (9, 2923, 7.541), (9, 713, 7.541), (9, 357, 7.541), (9, 6351, 7.541), (9, 4059, 7.541), (9, 376, 7.541), (9, 3958, 7.541), (9, 1004, 7.541), (9, 1887, 7.541), (9, 4472, 7.541), (9, 2963, 7.541), (9, 527, 7.541), (9, 1117, 7.541), (9, 1118, 7.541), (9, 846, 7.541), (9, 1965, 7.541),

(9, 4382, 7.541), (9, 1195, 7.541), (9, 232, 7.541), (9, 94, 7.541), (9, 79, 7.541), (9, 177, 7.541), (9, 1840, 7.541), (9, 481, 7.541), (9, 9062, 7.541), (9, 6746, 7.541), (9, 9314, 7.541), (9, 6594, 7.541), (9, 8129, 7.541), (9, 9756, 7.541), (9, 7769, 7.541), (9, 10067, 7.541), (9, 5081, 7.541), (9, 4835, 7.541), (9, 4177, 7.541), (9, 2164, 7.541), (9, 5177, 7.541), (9, 1726, 7.541), (9, 552, 7.541), (9, 874, 7.541), (9, 3230, 7.541), (9, 408, 7.541), (9, 387, 7.541), (9, 6895, 7.541), (9, 452, 7.541), (9, 450, 7.541), (9, 451, 7.541), (9, 7791, 7.541), (9, 2593, 7.541), (9, 3782, 7.541), (9, 3783, 7.541), (9, 4280, 7.541), (9, 5204, 7.541), (9, 5205, 7.541), (9, 1691, 7.541), (9, 6205, 7.541), (9, 1579, 7.541), (9, 3572, 7.541), (9, 98, 7.541), (9, 3342, 7.541), (9, 4081, 7.541), (9, 6512, 7.541), (9, 4155, 7.541), (9, 5341, 7.541), (9, 6007, 7.541), (9, 8769, 7.541), (9, 437, 7.541), (9, 65, 7.541), (9, 339, 7.541), (9, 5681, 7.541), (9, 7311, 7.541), (9, 6211, 7.541), (9, 4224, 7.541), (9, 27, 7.541), (9, 2129, 7.541), (9, 969, 7.541), (9, 6956, 7.541), (9, 861, 7.541), (9, 3712, 7.541), (9, 5300, 7.541), (9, 4975, 7.541), (9, 6747, 7.541), (9, 2581, 7.541), (9, 5530, 7.541), (9, 5682, 7.541), (9, 6500, 7.541), (9, 3091, 7.541), (9, 67, 7.541), (9, 149, 7.541), (9, 5909, 7.541), (9, 371, 7.541), (9, 372, 7.541), (9, 1313, 7.541), (9, 2961, 7.541), (9, 1132, 7.541), (20, 48, 7.541), (20, 53, 7.541), (20, 47, 7.541), (20, 591, 7.541), (20, 54, 7.541), (20, 57, 7.541), (20, 68, 7.541), (20, 1222, 7.541), (20, 1689, 7.541), (20, 59, 7.541), (20, 60, 7.541), (20, 61, 7.541), (20, 63, 7.541), (20, 1361, 7.541), (20, 227, 7.541), (20, 120, 7.541), (20, 73, 7.541), (20, 1015, 7.541), (20, 72, 7.541), (20, 122, 7.541), (20, 132, 7.541), (20, 1571, 7.541), (20, 241, 7.541), (20, 268, 7.541), (20, 133, 7.541), (20, 267, 7.541), (20, 16, 7.541), (20, 15809, 7.541), (20, 5630, 7.541), (20, 135, 7.541), (20, 256, 7.541), (20, 248, 7.541), (20, 185, 7.541), (20, 249, 7.541), (20, 93, 7.541), (20, 147, 7.541), (20, 32281, 7.541), (20, 412, 7.541), (20, 189, 7.541), (20, 190, 7.541), (20, 191, 7.541), (20, 192, 7.541), (20, 2034, 7.541), (20, 330, 7.541), (20, 19, 7.541), (20, 877, 7.541), (20, 936, 7.541), (20, 30, 7.541), (20, 31, 7.541), (20, 32, 7.541), (20, 1210, 7.541), (20, 195, 7.541), (20, 196, 7.541), (20, 12355, 7.541), (20, 2966, 7.541), (20, 322, 7.541), (20, 210, 7.541), (20, 710, 7.541), (20, 401, 7.541), (20, 44, 7.541), (20, 2476, 7.541), (20, 24, 7.541), (20, 400, 7.541), (20, 2104, 7.541), (20, 90, 7.541), (20, 25777, 7.541), (20, 169, 7.541), (20, 2001, 7.541), (20, 182, 7.541), (20, 2236, 7.541), (20, 10162, 7.541), (20, 7, 7.541), (20, 1562, 7.541), (20, 15, 7.541), (20, 1699, 7.541), (20, 2605, 7.541), (20, 857, 7.541), (20, 66, 7.541), (20, 150, 7.541), (20, 2904, 7.541), (20, 235, 7.541), (20, 2026, 7.541), (20, 777, 7.541), (20, 1530, 7.541), (20, 145, 7.541), (20, 1604, 7.541), (20, 157, 7.541), (20, 323, 7.541), (20, 298, 7.541), (20, 5525, 7.541), (20, 6682, 7.541), (20, 11759, 7.541), (20, 101, 7.541), (20, 102, 7.541), (20, 22199, 7.541), (20, 4744, 7.541), (20, 25013, 7.541), (20, 22729, 7.541), (20, 27655, 7.541), (20, 6547, 7.541), (20, 9989, 7.541), (20, 11433, 7.541), (20, 11111, 7.541), (20, 24833, 7.541), (20, 21995, 7.541), (20, 9919, 7.541),

(20, 12967, 7.541), (20, 477, 7.541), (20, 9736, 7.541), (20, 5342, 7.541), (20, 2251, 7.541), (20, 3901, 7.541), (20, 6347, 7.541), (20, 7674, 7.541), (20, 10030, 7.541), (20, 2986, 7.541), (20, 22789, 7.541), (20, 10396, 7.541), (20, 13535, 7.541), (20, 20787, 7.541), (20, 7059, 7.541), (20, 10490, 7.541), (20, 31043, 7.541), (20, 10719, 7.541), (20, 14967, 7.541), (20, 6772, 7.541), (20, 8514, 7.541), (20, 14345, 7.541), (20, 31478, 7.541), (20, 1536, 7.541), (20, 10163, 7.541), (20, 12293, 7.541), (20, 5356, 7.541), (20, 10578, 7.541), (20, 28999, 7.541), (20, 10800, 7.541), (20, 14397, 7.541), (20, 4186, 7.541), (20, 14741, 7.541), (20, 18671, 7.541), (20, 2167, 7.541), (20, 11703, 7.541), (20, 12549, 7.541), (20, 16592, 7.541), (20, 35849, 7.541), (20, 15583, 7.541), (20, 28223, 7.541), (20, 8086, 7.541), (20, 9330, 7.541), (20, 28121, 7.541), (20, 2924, 7.541), (20, 1250, 7.541), (20, 2759, 7.541), (20, 3784, 7.541), (20, 3785, 7.541), (20, 8536, 7.541), (20, 9367, 7.541), (20, 908, 7.541), (20, 123, 7.541), (20, 4725, 7.541), (20, 25183, 7.541), (20, 17895, 7.541), (20, 8425, 7.541), (20, 10793, 7.541), (20, 134, 7.541), (20, 13161, 7.541), (20, 31859, 7.541), (20, 20583, 7.541), (20, 32935, 7.541), (20, 28891, 7.541), (20, 587, 7.541), (20, 15225, 7.541), (20, 8630, 7.541), (20, 8074, 7.541), (20, 9515, 7.541), (20, 934, 7.541), (20, 1889, 7.541), (20, 9041, 7.541), (20, 3731, 7.541), (20, 228, 7.541), (20, 3092, 7.541), (20, 4814, 7.541), (20, 14467, 7.541), (20, 5680, 7.541), (20, 6862, 7.541), (20, 7054, 7.541), (20, 813, 7.541), (20, 467, 7.541), (20, 379, 7.541), (20, 28171, 7.541), (20, 1292, 7.541), (20, 7647, 7.541), (20, 9513, 7.541), (20, 1498, 7.541), (20, 4901, 7.541), (20, 11285, 7.541), (20, 1686, 7.541), (20, 2889, 7.541), (20, 762, 7.541), (20, 834, 7.541), (20, 11633, 7.541), (20, 405, 7.541), (20, 11241, 7.541), (20, 4981, 7.541), (20, 8915, 7.541), (20, 4182, 7.541), (20, 7338, 7.541), (20, 6573, 7.541), (20, 6880, 7.541), (20, 10155, 7.541), (20, 223, 7.541), (20, 225, 7.541), (20, 6033, 7.541), (20, 22297, 7.541), (20, 28701, 7.541), (20, 4192, 7.541), (20, 11617, 7.541), (20, 8277, 7.541), (20, 7088, 7.541), (20, 468, 7.541), (20, 18679, 7.541), (20, 22535, 7.541), (20, 801, 7.541), (20, 237, 7.541), (20, 4898, 7.541), (20, 6707, 7.541), (20, 18153, 7.541), (20, 17265, 7.541), (20, 23321, 7.541), (20, 18115, 7.541), (20, 14513, 7.541), (20, 20785, 7.541), (20, 23755, 7.541), (20, 442, 7.541), (20, 19815, 7.541), (20, 20507, 7.541), (20, 13601, 7.541), (20, 23281, 7.541), (20, 2993, 7.541), (20, 4214, 7.541), (20, 64, 7.541), (20, 4063, 7.541), (20, 5355, 7.541), (20, 154, 7.541), (20, 3588, 7.541), (20, 2418, 7.541), (20, 21881, 7.541), (20, 9941, 7.541), (20, 6213, 7.541), (20, 4654, 7.541), (20, 8937, 7.541), (20, 22319, 7.541), (20, 27899, 7.541), (20, 3167, 7.541), (20, 2787, 7.541), (20, 530, 7.541), (20, 740, 7.541), (20, 14713, 7.541), (20, 3503, 7.541), (20, 6045, 7.541), (20, 9656, 7.541), (20, 355, 7.541), (20, 4722, 7.541), (20, 534, 7.541), (20, 3470, 7.541), (20, 513, 7.541), (20, 14227, 7.541), (20, 3457, 7.541), (20, 4752, 7.541), (20, 4181, 7.541), (20, 10620, 7.541), (20, 2923, 7.541), (20, 713, 7.541), (20, 357, 7.541), (20, 6351, 7.541), (20, 4059, 7.541), (20, 376, 7.541), (20, 3958, 7.541), (20, 1004, 7.541), (20, 1887, 7.541), (20, 4472, 7.541), (20, 2963,

7.541), (20, 527, 7.541), (20, 1117, 7.541), (20, 1118, 7.541), (20, 846, 7.541), (20, 1965, 7.541), (20, 4382, 7.541), (20, 1195, 7.541), (20, 232, 7.541), (20, 94, 7.541), (20, 79, 7.541), (20, 177, 7.541), (20, 1840, 7.541), (20, 481, 7.541), (20, 9062, 7.541), (20, 6746, 7.541), (20, 9314, 7.541), (20, 6594, 7.541), (20, 8129, 7.541), (20, 9756, 7.541), (20, 7769, 7.541), (20, 10067, 7.541), (20, 5081, 7.541), (20, 4835, 7.541), (20, 4177, 7.541), (20, 2164, 7.541), (20, 5177, 7.541), (20, 1726, 7.541), (20, 552, 7.541), (20, 874, 7.541), (20, 3230, 7.541), (20, 408, 7.541), (20, 387, 7.541), (20, 6895, 7.541), (20, 452, 7.541), (20, 450, 7.541), (20, 451, 7.541), (20, 7791, 7.541), (20, 2593, 7.541), (20, 3782, 7.541), (20, 3783, 7.541), (20, 4280, 7.541), (20, 5204, 7.541), (20, 5205, 7.541), (20, 1691, 7.541), (20, 6205, 7.541), (20, 1579, 7.541), (20, 3572, 7.541), (20, 98, 7.541), (20, 3342, 7.541), (20, 4081, 7.541), (20, 6512, 7.541), (20, 4155, 7.541), (20, 5341, 7.541), (20, 6007, 7.541), (20, 8769, 7.541), (20, 437, 7.541), (20, 65, 7.541), (20, 339, 7.541), (20, 5681, 7.541), (20, 7311, 7.541), (20, 6211, 7.541), (20, 4224, 7.541), (20, 27, 7.541), (20, 2129, 7.541), (20, 969, 7.541), (20, 6956, 7.541), (20, 861, 7.541), (20, 3712, 7.541), (20, 5300, 7.541), (20, 4975, 7.541), (20, 6747, 7.541), (20, 2581, 7.541), (20, 5530, 7.541), (20, 5682, 7.541), (20, 6500, 7.541), (20, 3091, 7.541), (20, 67, 7.541), (20, 149, 7.541), (20, 5909, 7.541), (20, 371, 7.541), (20, 372, 7.541), (20, 1313, 7.541), (20, 2961, 7.541), (20, 1132, 7.541), (23, 48, 7.541), (23, 53, 7.541), (23, 47, 7.541), (23, 591, 7.541), (23, 54, 7.541), (23, 1222, 7.541), (23, 1689, 7.541), (23, 59, 7.541), (23, 60, 7.541), (23, 61, 7.541), (23, 63, 7.541), (23, 227, 7.541), (23, 120, 7.541), (23, 71, 7.541), (23, 1015, 7.541), (23, 72, 7.541), (23, 122, 7.541), (23, 2890, 7.541), (23, 239, 7.541), (23, 384, 7.541), (23, 132, 7.541), (23, 1571, 7.541), (23, 268, 7.541), (23, 133, 7.541), (23, 16, 7.541), (23, 5630, 7.541), (23, 135, 7.541), (23, 256, 7.541), (23, 431, 7.541), (23, 248, 7.541), (23, 249, 7.541), (23, 33, 7.541), (23, 147, 7.541), (23, 32281, 7.541), (23, 486, 7.541), (23, 412, 7.541), (23, 433, 7.541), (23, 416, 7.541), (23, 191, 7.541), (23, 192, 7.541), (23, 2034, 7.541), (23, 330, 7.541), (23, 585, 7.541), (23, 164, 7.541), (23, 19, 7.541), (23, 457, 7.541), (23, 877, 7.541), (23, 597, 7.541), (23, 31, 7.541), (23, 32, 7.541), (23, 1210, 7.541), (23, 1698, 7.541), (23, 30276, 7.541), (23, 195, 7.541), (23, 196, 7.541), (23, 12355, 7.541), (23, 2966, 7.541), (23, 853, 7.541), (23, 1943, 7.541), (23, 322, 7.541), (23, 210, 7.541), (23, 710, 7.541), (23, 45, 7.541), (23, 401, 7.541), (23, 44, 7.541), (23, 2476, 7.541), (23, 24, 7.541), (23, 400, 7.541), (23, 1827, 7.541), (23, 199, 7.541), (23, 2104, 7.541), (23, 90, 7.541), (23, 25777, 7.541), (23, 169, 7.541), (23, 849, 7.541), (23, 174, 7.541), (23, 182, 7.541), (23, 2236, 7.541), (23, 523, 7.541), (23, 10162, 7.541), (23, 7, 7.541), (23, 202, 7.541), (23, 1562, 7.541), (23, 392, 7.541), (23, 1699, 7.541), (23, 2605, 7.541), (23, 66, 7.541), (23, 235, 7.541), (23, 3002, 7.541), (23, 1530, 7.541), (23, 145, 7.541), (23, 157, 7.541), (23, 323, 7.541), (23, 298, 7.541), (23, 6682, 7.541), (23, 11759, 7.541), (23, 101, 7.541), (23, 102,

7.541), (23, 4744, 7.541), (23, 25013, 7.541), (23, 9989, 7.541), (23, 11433, 7.541), (23, 11111, 7.541), (23, 24833, 7.541), (23, 21995, 7.541), (23, 12967, 7.541), (23, 477, 7.541), (23, 9736, 7.541), (23, 5342, 7.541), (23, 3901, 7.541), (23, 6347, 7.541), (23, 7674, 7.541), (23, 10030, 7.541), (23, 22789, 7.541), (23, 10396, 7.541), (23, 13535, 7.541), (23, 7059, 7.541), (23, 31043, 7.541), (23, 31964, 7.541), (23, 33486, 7.541), (23, 36456, 7.541), (23, 10719, 7.541), (23, 14967, 7.541), (23, 6772, 7.541), (23, 8514, 7.541), (23, 31478, 7.541), (23, 1536, 7.541), (23, 12293, 7.541), (23, 5356, 7.541), (23, 10578, 7.541), (23, 28999, 7.541), (23, 10800, 7.541), (23, 14397, 7.541), (23, 4186, 7.541), (23, 14741, 7.541), (23, 18671, 7.541), (23, 2167, 7.541), (23, 11703, 7.541), (23, 12549, 7.541), (23, 16592, 7.541), (23, 35849, 7.541), (23, 15583, 7.541), (23, 28223, 7.541), (23, 8086, 7.541), (23, 9330, 7.541), (23, 2924, 7.541), (23, 1250, 7.541), (23, 2759, 7.541), (23, 3784, 7.541), (23, 3785, 7.541), (23, 8536, 7.541), (23, 18507, 7.541), (23, 908, 7.541), (23, 123, 7.541), (23, 17895, 7.541), (23, 31859, 7.541), (23, 32935, 7.541), (23, 28891, 7.541), (23, 587, 7.541), (23, 15225, 7.541), (23, 8630, 7.541), (23, 8074, 7.541), (23, 9515, 7.541), (23, 934, 7.541), (23, 1889, 7.541), (23, 3731, 7.541), (23, 228, 7.541), (23, 3092, 7.541), (23, 4814, 7.541), (23, 14467, 7.541), (23, 5680, 7.541), (23, 6862, 7.541), (23, 7054, 7.541), (23, 379, 7.541), (23, 10087, 7.541), (23, 11741, 7.541), (23, 395, 7.541), (23, 578, 7.541), (23, 14719, 7.541), (23, 4282, 7.541), (23, 572, 7.541), (23, 34599, 7.541), (23, 512, 7.541), (23, 3655, 7.541), (23, 4437, 7.541), (23, 594, 7.541), (23, 4477, 7.541), (23, 29803, 7.541), (23, 528, 7.541), (23, 9253, 7.541), (23, 16524, 7.541), (23, 3167, 7.541), (23, 2787, 7.541), (23, 530, 7.541), (23, 740, 7.541), (23, 14713, 7.541), (23, 3503, 7.541), (23, 6045, 7.541), (23, 9656, 7.541), (23, 355, 7.541), (23, 4722, 7.541), (23, 534, 7.541), (23, 3470, 7.541), (23, 513, 7.541), (23, 14227, 7.541), (23, 3457, 7.541), (23, 4752, 7.541), (23, 4181, 7.541), (23, 10620, 7.541), (23, 2923, 7.541), (23, 713, 7.541), (23, 357, 7.541), (23, 6351, 7.541), (23, 4059, 7.541), (23, 376, 7.541), (23, 3958, 7.541), (23, 1004, 7.541), (23, 1887, 7.541), (23, 4472, 7.541), (23, 2963, 7.541), (23, 527, 7.541), (23, 1117, 7.541), (23, 1118, 7.541), (23, 846, 7.541), (23, 1965, 7.541), (23, 4382, 7.541), (23, 1195, 7.541), (23, 232, 7.541), (23, 94, 7.541), (23, 79, 7.541), (23, 177, 7.541), (23, 1840, 7.541), (23, 481, 7.541), (23, 9062, 7.541), (23, 6746, 7.541), (23, 9314, 7.541), (23, 6594, 7.541), (23, 8129, 7.541), (23, 9756, 7.541), (23, 7769, 7.541), (23, 10067, 7.541), (23, 5081, 7.541), (23, 4835, 7.541), (23, 4177, 7.541), (23, 2164, 7.541), (23, 5177, 7.541), (23, 1726, 7.541), (23, 552, 7.541), (23, 874, 7.541), (23, 3230, 7.541), (23, 408, 7.541), (23, 387, 7.541), (23, 6895, 7.541), (23, 452, 7.541), (23, 450, 7.541), (23, 451, 7.541), (23, 7791, 7.541), (23, 2593, 7.541), (23, 3782, 7.541), (23, 3783, 7.541), (23, 4280, 7.541), (23, 5204, 7.541), (23, 5205, 7.541), (23, 1691, 7.541), (23, 6205, 7.541), (23, 1579, 7.541), (23, 3572, 7.541), (23, 98, 7.541), (23, 3342, 7.541), (23, 4081, 7.541), (23, 6512, 7.541), (23, 4155, 7.541), (23, 5341, 7.541), (23, 6007, 7.541), (23, 8769,

7.541), (23, 437, 7.541), (23, 65, 7.541), (23, 339, 7.541), (23, 5681, 7.541), (23, 7311, 7.541), (23, 6211, 7.541), (23, 4224, 7.541), (23, 27, 7.541), (23, 2129, 7.541), (23, 969, 7.541), (23, 6956, 7.541), (23, 861, 7.541), (23, 3712, 7.541), (23, 5300, 7.541), (23, 4975, 7.541), (23, 6747, 7.541), (23, 2581, 7.541), (23, 5530, 7.541), (23, 5682, 7.541), (23, 6500, 7.541), (23, 3091, 7.541), (23, 67, 7.541), (23, 149, 7.541), (23, 5909, 7.541), (23, 371, 7.541), (23, 372, 7.541), (23, 1313, 7.541), (23, 2961, 7.541), (23, 1132, 7.541), (37, 21, 7.541), (37, 48, 7.541), (37, 53, 7.541), (37, 47, 7.541), (37, 54, 7.541), (37, 57, 7.541), (37, 68, 7.541), (37, 889, 7.541), (37, 1519, 7.541), (37, 1689, 7.541), (37, 1575, 7.541), (37, 1, 7.541), (37, 5, 7.541), (37, 2025, 7.541), (37, 63, 7.541), (37, 356, 7.541), (37, 317, 7.541), (37, 1361, 7.541), (37, 227, 7.541), (37, 1015, 7.541), (37, 122, 7.541), (37, 121, 7.541), (37, 2890, 7.541), (37, 239, 7.541), (37, 384, 7.541), (37, 240, 7.541), (37, 2508, 7.541), (37, 132, 7.541), (37, 1571, 7.541), (37, 241, 7.541), (37, 268, 7.541), (37, 245, 7.541), (37, 133, 7.541), (37, 267, 7.541), (37, 16, 7.541), (37, 263, 7.541), (37, 5258, 7.541), (37, 270, 7.541), (37, 135, 7.541), (37, 256, 7.541), (37, 431, 7.541), (37, 136, 7.541), (37, 11061, 7.541), (37, 138, 7.541), (37, 139, 7.541), (37, 137, 7.541), (37, 248, 7.541), (37, 257, 7.541), (37, 185, 7.541), (37, 249, 7.541), (37, 33, 7.541), (37, 93, 7.541), (37, 32281, 7.541), (37, 486, 7.541), (37, 412, 7.541), (37, 43, 7.541), (37, 433, 7.541), (37, 416, 7.541), (37, 97, 7.541), (37, 189, 7.541), (37, 190, 7.541), (37, 191, 7.541), (37, 192, 7.541), (37, 585, 7.541), (37, 164, 7.541), (37, 19, 7.541), (37, 457, 7.541), (37, 20, 7.541), (37, 936, 7.541), (37, 1735, 7.541), (37, 597, 7.541), (37, 30, 7.541), (37, 31, 7.541), (37, 32, 7.541), (37, 1210, 7.541), (37, 30276, 7.541), (37, 12355, 7.541), (37, 2966, 7.541), (37, 1943, 7.541), (37, 210, 7.541), (37, 45, 7.541), (37, 401, 7.541), (37, 44, 7.541), (37, 205, 7.541), (37, 2476, 7.541), (37, 24, 7.541), (37, 400, 7.541), (37, 1827, 7.541), (37, 2104, 7.541), (37, 1559, 7.541), (37, 90, 7.541), (37, 25777, 7.541), (37, 169, 7.541), (37, 849, 7.541), (37, 2001, 7.541), (37, 174, 7.541), (37, 523, 7.541), (37, 6, 7.541), (37, 10162, 7.541), (37, 202, 7.541), (37, 1562, 7.541), (37, 392, 7.541), (37, 269, 7.541), (37, 15, 7.541), (37, 2605, 7.541), (37, 22, 7.541), (37, 857, 7.541), (37, 66, 7.541), (37, 2904, 7.541), (37, 1535, 7.541), (37, 235, 7.541), (37, 3002, 7.541), (37, 2026, 7.541), (37, 777, 7.541), (37, 1530, 7.541), (37, 1604, 7.541), (37, 157, 7.541), (37, 323, 7.541), (37, 298, 7.541), (37, 5525, 7.541), (37, 6682, 7.541), (37, 11759, 7.541), (37, 101, 7.541), (37, 22199, 7.541), (37, 4744, 7.541), (37, 25013, 7.541), (37, 22729, 7.541), (37, 27655, 7.541), (37, 6547, 7.541), (37, 9989, 7.541), (37, 11433, 7.541), (37, 11111, 7.541), (37, 24833, 7.541), (37, 21995, 7.541), (37, 9919, 7.541), (37, 12967, 7.541), (37, 477, 7.541), (37, 9736, 7.541), (37, 5342, 7.541), (37, 2251, 7.541), (37, 3901, 7.541), (37, 6347, 7.541), (37, 7674, 7.541), (37, 10030, 7.541), (37, 2986, 7.541), (37, 22789, 7.541), (37, 10396, 7.541), (37, 13535, 7.541), (37, 20787, 7.541), (37, 7059, 7.541), (37, 10490, 7.541), (37, 31043, 7.541), (37, 31964, 7.541), (37, 33486, 7.541), (37, 36456, 7.541),

(37, 10719, 7.541), (37, 14967, 7.541), (37, 6772, 7.541), (37, 8514, 7.541), (37, 14345, 7.541), (37, 31478, 7.541), (37, 1536, 7.541), (37, 10163, 7.541), (37, 12293, 7.541), (37, 5356, 7.541), (37, 10578, 7.541), (37, 28999, 7.541), (37, 10800, 7.541), (37, 14397, 7.541), (37, 4186, 7.541), (37, 14741, 7.541), (37, 18671, 7.541), (37, 2167, 7.541), (37, 11703, 7.541), (37, 12549, 7.541), (37, 16592, 7.541), (37, 35849, 7.541), (37, 15583, 7.541), (37, 28223, 7.541), (37, 8086, 7.541), (37, 9330, 7.541), (37, 28121, 7.541), (37, 2924, 7.541), (37, 1250, 7.541), (37, 2759, 7.541), (37, 3784, 7.541), (37, 3785, 7.541), (37, 8536, 7.541), (37, 18507, 7.541), (37, 9367, 7.541), (37, 5114, 7.541), (37, 908, 7.541), (37, 430, 7.541), (37, 123, 7.541), (37, 4725, 7.541), (37, 25183, 7.541), (37, 8425, 7.541), (37, 10793, 7.541), (37, 134, 7.541), (37, 13161, 7.541), (37, 31859, 7.541), (37, 20583, 7.541), (37, 32935, 7.541), (37, 28891, 7.541), (37, 15225, 7.541), (37, 8630, 7.541), (37, 8074, 7.541), (37, 9515, 7.541), (37, 934, 7.541), (37, 1889, 7.541), (37, 9041, 7.541), (37, 228, 7.541), (37, 4814, 7.541), (37, 14467, 7.541), (37, 5680, 7.541), (37, 6862, 7.541), (37, 813, 7.541), (37, 467, 7.541), (37, 379, 7.541), (37, 790, 7.541), (37, 10087, 7.541), (37, 11741, 7.541), (37, 395, 7.541), (37, 578, 7.541), (37, 14719, 7.541), (37, 4282, 7.541), (37, 572, 7.541), (37, 34599, 7.541), (37, 512, 7.541), (37, 3655, 7.541), (37, 4437, 7.541), (37, 594, 7.541), (37, 4477, 7.541), (37, 29803, 7.541), (37, 528, 7.541), (37, 166, 7.541), (37, 9253, 7.541), (37, 16524, 7.541), (37, 6702, 7.541), (37, 1482, 7.541), (37, 28171, 7.541), (37, 1292, 7.541), (37, 7647, 7.541), (37, 9513, 7.541), (37, 1498, 7.541), (37, 4901, 7.541), (37, 11285, 7.541), (37, 1686, 7.541), (37, 2889, 7.541), (37, 762, 7.541), (37, 834, 7.541), (37, 11633, 7.541), (37, 405, 7.541), (37, 11241, 7.541), (37, 4981, 7.541), (37, 8915, 7.541), (37, 4182, 7.541), (37, 7338, 7.541), (37, 6573, 7.541), (37, 6880, 7.541), (37, 10155, 7.541), (37, 223, 7.541), (37, 225, 7.541), (37, 6033, 7.541), (37, 22297, 7.541), (37, 28701, 7.541), (37, 4192, 7.541), (37, 11617, 7.541), (37, 8277, 7.541), (37, 7088, 7.541), (37, 468, 7.541), (37, 18679, 7.541), (37, 22535, 7.541), (37, 801, 7.541), (37, 237, 7.541), (37, 4898, 7.541), (37, 6707, 7.541), (37, 18153, 7.541), (37, 17265, 7.541), (37, 23321, 7.541), (37, 18115, 7.541), (37, 14513, 7.541), (37, 20785, 7.541), (37, 23755, 7.541), (37, 442, 7.541), (37, 19815, 7.541), (37, 13601, 7.541), (37, 23281, 7.541), (37, 2993, 7.541), (37, 4214, 7.541), (37, 64, 7.541), (37, 4063, 7.541), (37, 5355, 7.541), (37, 154, 7.541), (37, 3588, 7.541), (37, 2418, 7.541), (37, 9941, 7.541), (37, 6213, 7.541), (37, 4654, 7.541), (37, 8937, 7.541), (37, 22319, 7.541), (37, 27899, 7.541), (37, 4181, 7.541), (37, 10620, 7.541), (37, 2923, 7.541), (37, 713, 7.541), (37, 357, 7.541), (37, 6351, 7.541), (37, 4059, 7.541), (37, 376, 7.541), (37, 3958, 7.541), (37, 1004, 7.541), (37, 1887, 7.541), (37, 4472, 7.541), (37, 2963, 7.541), (37, 527, 7.541), (37, 1117, 7.541), (37, 1118, 7.541), (37, 846, 7.541), (37, 1965, 7.541), (37, 4382, 7.541), (37, 1195, 7.541), (37, 232, 7.541), (37, 94, 7.541), (37, 79, 7.541), (37, 177, 7.541), (37, 1840, 7.541), (37, 481, 7.541), (37, 9062, 7.541), (37, 6746, 7.541), (37, 9314, 7.541), (37, 6594, 7.541), (37,

8129, 7.541), (37, 9756, 7.541), (37, 7769, 7.541), (37, 10067, 7.541), (37, 5081, 7.541), (37, 4835, 7.541), (37, 4177, 7.541), (37, 2164, 7.541), (37, 5177, 7.541), (37, 1726, 7.541), (37, 552, 7.541), (37, 874, 7.541), (37, 3230, 7.541), (37, 408, 7.541), (37, 387, 7.541), (37, 6895, 7.541), (37, 452, 7.541), (37, 450, 7.541), (37, 451, 7.541), (37, 7791, 7.541), (37, 2593, 7.541), (37, 3782, 7.541), (37, 3783, 7.541), (37, 4280, 7.541), (37, 5204, 7.541), (37, 5205, 7.541), (37, 1691, 7.541), (37, 6205, 7.541), (37, 1579, 7.541), (37, 3572, 7.541), (37, 98, 7.541), (37, 3342, 7.541), (37, 4081, 7.541), (37, 6512, 7.541), (37, 4155, 7.541), (37, 5341, 7.541), (37, 6007, 7.541), (37, 8769, 7.541), (37, 437, 7.541), (37, 65, 7.541), (37, 339, 7.541), (37, 5681, 7.541), (37, 7311, 7.541), (37, 6211, 7.541), (37, 4224, 7.541), (37, 27, 7.541), (37, 2129, 7.541), (37, 969, 7.541), (37, 6956, 7.541), (37, 861, 7.541), (37, 3712, 7.541), (37, 5300, 7.541), (37, 4975, 7.541), (37, 6747, 7.541), (37, 2581, 7.541), (37, 5530, 7.541), (37, 5682, 7.541), (37, 6500, 7.541), (37, 3091, 7.541), (37, 67, 7.541), (37, 149, 7.541), (37, 5909, 7.541), (37, 371, 7.541), (37, 372, 7.541), (37, 1313, 7.541), (37, 2961, 7.541), (37, 1132, 7.541), (47, 21, 7.541), (47, 47, 7.541), (47, 591, 7.541), (47, 54, 7.541), (47, 57, 7.541), (47, 68, 7.541), (47, 889, 7.541), (47, 1519, 7.541), (47, 1222, 7.541), (47, 60, 7.541), (47, 1818, 7.541), (47, 61, 7.541), (47, 2025, 7.541), (47, 63, 7.541), (47, 356, 7.541), (47, 1361, 7.541), (47, 120, 7.541), (47, 73, 7.541), (47, 1015, 7.541), (47, 2890, 7.541), (47, 239, 7.541), (47, 384, 7.541), (47, 240, 7.541), (47, 2508, 7.541), (47, 132, 7.541), (47, 1571, 7.541), (47, 241, 7.541), (47, 268, 7.541), (47, 133, 7.541), (47, 267, 7.541), (47, 263, 7.541), (47, 5258, 7.541), (47, 15809, 7.541), (47, 270, 7.541), (47, 5630, 7.541), (47, 135, 7.541), (47, 431, 7.541), (47, 136, 7.541), (47, 11061, 7.541), (47, 138, 7.541), (47, 139, 7.541), (47, 137, 7.541), (47, 248, 7.541), (47, 257, 7.541), (47, 185, 7.541), (47, 249, 7.541), (47, 33, 7.541), (47, 486, 7.541), (47, 412, 7.541), (47, 43, 7.541), (47, 416, 7.541), (47, 97, 7.541), (47, 2034, 7.541), (47, 330, 7.541), (47, 585, 7.541), (47, 19, 7.541), (47, 457, 7.541), (47, 877, 7.541), (47, 936, 7.541), (47, 597, 7.541), (47, 1210, 7.541), (47, 1698, 7.541), (47, 195, 7.541), (47, 196, 7.541), (47, 12355, 7.541), (47, 2966, 7.541), (47, 1943, 7.541), (47, 322, 7.541), (47, 2476, 7.541), (47, 400, 7.541), (47, 2104, 7.541), (47, 1559, 7.541), (47, 90, 7.541), (47, 25777, 7.541), (47, 169, 7.541), (47, 174, 7.541), (47, 182, 7.541), (47, 523, 7.541), (47, 6, 7.541), (47, 10162, 7.541), (47, 7, 7.541), (47, 202, 7.541), (47, 1562, 7.541), (47, 392, 7.541), (47, 15, 7.541), (47, 1699, 7.541), (47, 22, 7.541), (47, 857, 7.541), (47, 150, 7.541), (47, 2904, 7.541), (47, 235, 7.541), (47, 3002, 7.541), (47, 777, 7.541), (47, 145, 7.541), (47, 1604, 7.541), (47, 157, 7.541), (47, 323, 7.541), (47, 298, 7.541), (47, 5525, 7.541), (47, 6682, 7.541), (47, 11759, 7.541), (47, 102, 7.541), (47, 22199, 7.541), (47, 4744, 7.541), (47, 25013, 7.541), (47, 22729, 7.541), (47, 27655, 7.541), (47, 6547, 7.541), (47, 9989, 7.541), (47, 11433, 7.541), (47, 11111, 7.541), (47, 24833, 7.541), (47, 21995, 7.541), (47, 9919, 7.541),

(47, 12967, 7.541), (47, 477, 7.541), (47, 9736, 7.541), (47, 5342, 7.541), (47, 2251, 7.541), (47, 3901, 7.541), (47, 6347, 7.541), (47, 7674, 7.541), (47, 10030, 7.541), (47, 2986, 7.541), (47, 22789, 7.541), (47, 10396, 7.541), (47, 13535, 7.541), (47, 20787, 7.541), (47, 7059, 7.541), (47, 10490, 7.541), (47, 31043, 7.541), (47, 31964, 7.541), (47, 33486, 7.541), (47, 36456, 7.541), (47, 10719, 7.541), (47, 14967, 7.541), (47, 6772, 7.541), (47, 8514, 7.541), (47, 14345, 7.541), (47, 31478, 7.541), (47, 1536, 7.541), (47, 10163, 7.541), (47, 12293, 7.541), (47, 5356, 7.541), (47, 10578, 7.541), (47, 28999, 7.541), (47, 10800, 7.541), (47, 14397, 7.541), (47, 4186, 7.541), (47, 14741, 7.541), (47, 18671, 7.541), (47, 11703, 7.541), (47, 12549, 7.541), (47, 16592, 7.541), (47, 35849, 7.541), (47, 15583, 7.541), (47, 28223, 7.541), (47, 8086, 7.541), (47, 9330, 7.541), (47, 28121, 7.541), (47, 1250, 7.541), (47, 2759, 7.541), (47, 3784, 7.541), (47, 3785, 7.541), (47, 8536, 7.541), (47, 18507, 7.541), (47, 9367, 7.541), (47, 5114, 7.541), (47, 908, 7.541), (47, 123, 7.541), (47, 4725, 7.541), (47, 25183, 7.541), (47, 17895, 7.541), (47, 8425, 7.541), (47, 10793, 7.541), (47, 134, 7.541), (47, 13161, 7.541), (47, 31859, 7.541), (47, 20583, 7.541), (47, 32935, 7.541), (47, 28891, 7.541), (47, 15225, 7.541), (47, 8630, 7.541), (47, 8074, 7.541), (47, 9515, 7.541), (47, 1889, 7.541), (47, 9041, 7.541), (47, 3731, 7.541), (47, 3092, 7.541), (47, 4814, 7.541), (47, 14467, 7.541), (47, 6862, 7.541), (47, 7054, 7.541), (47, 813, 7.541), (47, 467, 7.541), (47, 790, 7.541), (47, 10087, 7.541), (47, 11741, 7.541), (47, 395, 7.541), (47, 14719, 7.541), (47, 4282, 7.541), (47, 572, 7.541), (47, 34599, 7.541), (47, 3655, 7.541), (47, 4437, 7.541), (47, 4477, 7.541), (47, 29803, 7.541), (47, 166, 7.541), (47, 9253, 7.541), (47, 16524, 7.541), (47, 11757, 7.541), (47, 6702, 7.541), (47, 1482, 7.541), (47, 28171, 7.541), (47, 1292, 7.541), (47, 7647, 7.541), (47, 9513, 7.541), (47, 1498, 7.541), (47, 4901, 7.541), (47, 11285, 7.541), (47, 1686, 7.541), (47, 2889, 7.541), (47, 762, 7.541), (47, 834, 7.541), (47, 11633, 7.541), (47, 405, 7.541), (47, 11241, 7.541), (47, 4981, 7.541), (47, 8915, 7.541), (47, 4182, 7.541), (47, 7338, 7.541), (47, 6573, 7.541), (47, 6880, 7.541), (47, 10155, 7.541), (47, 223, 7.541), (47, 225, 7.541), (47, 6033, 7.541), (47, 22297, 7.541), (47, 28701, 7.541), (47, 4192, 7.541), (47, 11617, 7.541), (47, 8277, 7.541), (47, 7088, 7.541), (47, 468, 7.541), (47, 18679, 7.541), (47, 22535, 7.541), (47, 801, 7.541), (47, 237, 7.541), (47, 4898, 7.541), (47, 6707, 7.541), (47, 18153, 7.541), (47, 17265, 7.541), (47, 23321, 7.541), (47, 18115, 7.541), (47, 14513, 7.541), (47, 20785, 7.541), (47, 23755, 7.541), (47, 442, 7.541), (47, 19815, 7.541), (47, 20507, 7.541), (47, 13601, 7.541), (47, 23281, 7.541), (47, 2993, 7.541), (47, 4214, 7.541), (47, 4063, 7.541), (47, 5355, 7.541), (47, 3588, 7.541), (47, 2418, 7.541), (47, 21881, 7.541), (47, 9941, 7.541), (47, 6213, 7.541), (47, 8937, 7.541), (47, 22319, 7.541), (47, 27899, 7.541), (47, 3167, 7.541), (47, 2787, 7.541), (47, 530, 7.541), (47, 740, 7.541), (47, 14713, 7.541), (47, 3503, 7.541), (47, 6045, 7.541), (47, 9656, 7.541), (47, 4722, 7.541), (47, 3470, 7.541), (47, 513, 7.541), (47, 14227, 7.541), (47, 3457, 7.541), (47, 4752, 7.541), (47,

9062, 7.541), (47, 6746, 7.541), (47, 9314, 7.541), (47, 6594, 7.541), (47, 8129, 7.541), (47, 9756, 7.541), (47, 7769, 7.541), (47, 10067, 7.541), (47, 5081, 7.541), (47, 4835, 7.541), (47, 4177, 7.541), (47, 2164, 7.541), (47, 5177, 7.541), (47, 1726, 7.541), (47, 552, 7.541), (47, 874, 7.541), (47, 3230, 7.541), (47, 408, 7.541), (47, 387, 7.541), (47, 6895, 7.541), (47, 452, 7.541), (47, 450, 7.541), (47, 451, 7.541), (47, 7791, 7.541), (47, 2593, 7.541), (47, 3782, 7.541), (47, 3783, 7.541), (47, 4280, 7.541), (47, 5204, 7.541), (47, 5205, 7.541), (47, 1691, 7.541), (47, 6205, 7.541), (47, 1579, 7.541), (47, 3572, 7.541), (47, 98, 7.541), (47, 3342, 7.541), (47, 4081, 7.541), (47, 6512, 7.541), (47, 4155, 7.541), (47, 5341, 7.541), (47, 6007, 7.541), (47, 8769, 7.541), (47, 437, 7.541), (47, 65, 7.541), (47, 339, 7.541), (47, 5681, 7.541), (47, 7311, 7.541), (47, 6211, 7.541), (47, 4224, 7.541), (47, 27, 7.541), (47, 2129, 7.541), (47, 969, 7.541), (47, 6956, 7.541), (47, 861, 7.541), (47, 3712, 7.541), (47, 5300, 7.541), (47, 4975, 7.541), (47, 6747, 7.541), (47, 2581, 7.541), (47, 5530, 7.541), (47, 5682, 7.541), (47, 6500, 7.541), (47, 3091, 7.541), (47, 67, 7.541), (47, 149, 7.541), (47, 5909, 7.541), (47, 371, 7.541), (47, 372, 7.541), (47, 1313, 7.541), (47, 2961, 7.541), (47, 1132, 7.541), (48, 21, 7.541), (48, 53, 7.541), (48, 47, 7.541), (48, 591, 7.541), (48, 68, 7.541), (48, 1222, 7.541), (48, 1, 7.541), (48, 5, 7.541), (48, 1361, 7.541), (48, 227, 7.541), (48, 120, 7.541), (48, 122, 7.541), (48, 384, 7.541), (48, 240, 7.541), (48, 2508, 7.541), (48, 241, 7.541), (48, 268, 7.541), (48, 245, 7.541), (48, 267, 7.541), (48, 16, 7.541), (48, 263, 7.541), (48, 5258, 7.541), (48, 15809, 7.541), (48, 135, 7.541), (48, 136, 7.541), (48, 11061, 7.541), (48, 138, 7.541), (48, 139, 7.541), (48, 137, 7.541), (48, 257, 7.541), (48, 185, 7.541), (48, 249, 7.541), (48, 33, 7.541), (48, 147, 7.541), (48, 32281, 7.541), (48, 486, 7.541), (48, 412, 7.541), (48, 43, 7.541), (48, 416, 7.541), (48, 189, 7.541), (48, 190, 7.541), (48, 191, 7.541), (48, 192, 7.541), (48, 2034, 7.541), (48, 585, 7.541), (48, 19, 7.541), (48, 877, 7.541), (48, 31, 7.541), (48, 30276, 7.541), (48, 12355, 7.541), (48, 1943, 7.541), (48, 322, 7.541), (48, 210, 7.541), (48, 710, 7.541), (48, 401, 7.541), (48, 2476, 7.541), (48, 199, 7.541), (48, 2104, 7.541), (48, 1559, 7.541), (48, 90, 7.541), (48, 16498, 7.541), (48, 25777, 7.541), (48, 2001, 7.541), (48, 182, 7.541), (48, 523, 7.541), (48, 6, 7.541), (48, 10162, 7.541), (48, 1562, 7.541), (48, 15, 7.541), (48, 2605, 7.541), (48, 22, 7.541), (48, 66, 7.541), (48, 235, 7.541), (48, 3002, 7.541), (48, 2026, 7.541), (48, 777, 7.541), (48, 145, 7.541), (48, 1604, 7.541), (48, 5525, 7.541), (48, 11759, 7.541), (48, 102, 7.541), (48, 22199, 7.541), (48, 4744, 7.541), (48, 25013, 7.541), (48, 22729, 7.541), (48, 27655, 7.541), (48, 9989, 7.541), (48, 11433, 7.541), (48, 11111, 7.541), (48, 24833, 7.541), (48, 21995, 7.541), (48, 9919, 7.541), (48, 12967, 7.541), (48, 9736, 7.541), (48, 7674, 7.541), (48, 10030, 7.541), (48, 22789, 7.541), (48, 10396, 7.541), (48, 13535, 7.541), (48, 20787, 7.541), (48, 7059, 7.541), (48, 10490, 7.541), (48, 31043, 7.541), (48, 31964, 7.541), (48, 33486, 7.541), (48, 36456, 7.541), (48, 10719, 7.541), (48, 14967, 7.541), (48, 6772,

7.541), (48, 8514, 7.541), (48, 14345, 7.541), (48, 31478, 7.541), (48, 10163, 7.541), (48, 12293, 7.541), (48, 10578, 7.541), (48, 28999, 7.541), (48, 10800, 7.541), (48, 14397, 7.541), (48, 14741, 7.541), (48, 18671, 7.541), (48, 11703, 7.541), (48, 12549, 7.541), (48, 16592, 7.541), (48, 35849, 7.541), (48, 15583, 7.541), (48, 28223, 7.541), (48, 9330, 7.541), (48, 28121, 7.541), (48, 2924, 7.541), (48, 3785, 7.541), (48, 8536, 7.541), (48, 18507, 7.541), (48, 25183, 7.541), (48, 17895, 7.541), (48, 8425, 7.541), (48, 10793, 7.541), (48, 13161, 7.541), (48, 31859, 7.541), (48, 20583, 7.541), (48, 32935, 7.541), (48, 28891, 7.541), (48, 15225, 7.541), (48, 8630, 7.541), (48, 8074, 7.541), (48, 9515, 7.541), (48, 934, 7.541), (48, 1889, 7.541), (48, 3731, 7.541), (48, 3092, 7.541), (48, 4814, 7.541), (48, 14467, 7.541), (48, 6862, 7.541), (48, 7054, 7.541), (48, 813, 7.541), (48, 379, 7.541), (48, 10087, 7.541), (48, 11741, 7.541), (48, 395, 7.541), (48, 578, 7.541), (48, 14719, 7.541), (48, 34599, 7.541), (48, 512, 7.541), (48, 29803, 7.541), (48, 528, 7.541), (48, 9253, 7.541), (48, 16524, 7.541), (48, 11757, 7.541), (48, 28171, 7.541), (48, 1292, 7.541), (48, 7647, 7.541), (48, 9513, 7.541), (48, 4901, 7.541), (48, 11285, 7.541), (48, 1686, 7.541), (48, 2889, 7.541), (48, 11633, 7.541), (48, 11241, 7.541), (48, 4981, 7.541), (48, 8915, 7.541), (48, 7338, 7.541), (48, 6880, 7.541), (48, 10155, 7.541), (48, 223, 7.541), (48, 6033, 7.541), (48, 22297, 7.541), (48, 28701, 7.541), (48, 4192, 7.541), (48, 11617, 7.541), (48, 8277, 7.541), (48, 18679, 7.541), (48, 22535, 7.541), (48, 4898, 7.541), (48, 6707, 7.541), (48, 18153, 7.541), (48, 17265, 7.541), (48, 23321, 7.541), (48, 18115, 7.541), (48, 14513, 7.541), (48, 20785, 7.541), (48, 23755, 7.541), (48, 19815, 7.541), (48, 20507, 7.541), (48, 13601, 7.541), (48, 23281, 7.541), (48, 2993, 7.541), (48, 4214, 7.541), (48, 4063, 7.541), (48, 154, 7.541), (48, 21881, 7.541), (48, 9941, 7.541), (48, 8937, 7.541), (48, 22319, 7.541), (48, 27899, 7.541), (48, 3167, 7.541), (48, 740, 7.541), (48, 14713, 7.541), (48, 3503, 7.541), (48, 6045, 7.541), (48, 9656, 7.541), (48, 4722, 7.541), (48, 534, 7.541), (48, 3470, 7.541), (48, 513, 7.541), (48, 14227, 7.541), (48, 3457, 7.541), (48, 4752, 7.541), (48, 4181, 7.541), (48, 10620, 7.541), (48, 2923, 7.541), (48, 376, 7.541), (48, 1004, 7.541), (48, 4472, 7.541), (48, 527, 7.541), (48, 1117, 7.541), (48, 1118, 7.541), (48, 846, 7.541), (48, 232, 7.541), (48, 79, 7.541), (48, 481, 7.541), (48, 371, 7.541), (48, 372, 7.541), (48, 1313, 7.541), (48, 2961, 7.541), (48, 1132, 7.541), (53, 21, 7.541), (53, 48, 7.541), (53, 53, 7.541), (53, 47, 7.541), (53, 591, 7.541), (53, 57, 7.541), (53, 68, 7.541), (53, 889, 7.541), (53, 1519, 7.541), (53, 1222, 7.541), (53, 1689, 7.541), (53, 59, 7.541), (53, 60, 7.541), (53, 1818, 7.541), (53, 1575, 7.541), (53, 61, 7.541), (53, 2025, 7.541), (53, 63, 7.541), (53, 317, 7.541), (53, 1361, 7.541), (53, 227, 7.541), (53, 120, 7.541), (53, 1015, 7.541), (53, 122, 7.541), (53, 121, 7.541), (53, 2890, 7.541), (53, 239, 7.541), (53, 384, 7.541), (53, 2508, 7.541), (53, 132, 7.541), (53, 1571, 7.541), (53, 241, 7.541), (53, 245, 7.541), (53, 133, 7.541), (53, 267, 7.541), (53, 16, 7.541), (53, 263, 7.541), (53, 5258, 7.541), (53, 15809, 7.541), (53, 270, 7.541), (53, 5630, 7.541),

(53, 135, 7.541), (53, 431, 7.541), (53, 136, 7.541), (53, 11061, 7.541), (53, 138, 7.541), (53, 139, 7.541), (53, 137, 7.541), (53, 248, 7.541), (53, 257, 7.541), (53, 185, 7.541), (53, 249, 7.541), (53, 33, 7.541), (53, 93, 7.541), (53, 32281, 7.541), (53, 486, 7.541), (53, 412, 7.541), (53, 43, 7.541), (53, 433, 7.541), (53, 416, 7.541), (53, 97, 7.541), (53, 189, 7.541), (53, 190, 7.541), (53, 191, 7.541), (53, 192, 7.541), (53, 2034, 7.541), (53, 585, 7.541), (53, 164, 7.541), (53, 19, 7.541), (53, 457, 7.541), (53, 877, 7.541), (53, 20, 7.541), (53, 936, 7.541), (53, 1735, 7.541), (53, 597, 7.541), (53, 30, 7.541), (53, 31, 7.541), (53, 32, 7.541), (53, 1210, 7.541), (53, 1698, 7.541), (53, 30276, 7.541), (53, 195, 7.541), (53, 12355, 7.541), (53, 2966, 7.541), (53, 853, 7.541), (53, 1943, 7.541), (53, 322, 7.541), (53, 210, 7.541), (53, 710, 7.541), (53, 45, 7.541), (53, 401, 7.541), (53, 2476, 7.541), (53, 24, 7.541), (53, 400, 7.541), (53, 1827, 7.541), (53, 199, 7.541), (53, 2104, 7.541), (53, 1559, 7.541), (53, 90, 7.541), (53, 16498, 7.541), (53, 25777, 7.541), (53, 849, 7.541), (53, 2001, 7.541), (53, 174, 7.541), (53, 182, 7.541), (53, 2236, 7.541), (53, 523, 7.541), (53, 6, 7.541), (53, 10162, 7.541), (53, 7, 7.541), (53, 202, 7.541), (53, 1562, 7.541), (53, 392, 7.541), (53, 269, 7.541), (53, 15, 7.541), (53, 1699, 7.541), (53, 2605, 7.541), (53, 22, 7.541), (53, 857, 7.541), (53, 66, 7.541), (53, 150, 7.541), (53, 2904, 7.541), (53, 1535, 7.541), (53, 235, 7.541), (53, 3002, 7.541), (53, 2026, 7.541), (53, 777, 7.541), (53, 1530, 7.541), (53, 145, 7.541), (53, 1604, 7.541), (53, 157, 7.541), (53, 298, 7.541), (53, 5525, 7.541), (53, 6682, 7.541), (53, 11759, 7.541), (53, 102, 7.541), (53, 22199, 7.541), (53, 4744, 7.541), (53, 25013, 7.541), (53, 22729, 7.541), (53, 27655, 7.541), (53, 6547, 7.541), (53, 9989, 7.541), (53, 11433, 7.541), (53, 11111, 7.541), (53, 24833, 7.541), (53, 21995, 7.541), (53, 9919, 7.541), (53, 12967, 7.541), (53, 477, 7.541), (53, 9736, 7.541), (53, 5342, 7.541), (53, 2251, 7.541), (53, 3901, 7.541), (53, 6347, 7.541), (53, 7674, 7.541), (53, 10030, 7.541), (53, 22789, 7.541), (53, 10396, 7.541), (53, 13535, 7.541), (53, 20787, 7.541), (53, 7059, 7.541), (53, 10490, 7.541), (53, 31043, 7.541), (53, 31964, 7.541), (53, 33486, 7.541), (53, 36456, 7.541), (53, 10719, 7.541), (53, 14967, 7.541), (53, 6772, 7.541), (53, 8514, 7.541), (53, 14345, 7.541), (53, 31478, 7.541), (53, 1536, 7.541), (53, 10163, 7.541), (53, 12293, 7.541), (53, 5356, 7.541), (53, 10578, 7.541), (53, 28999, 7.541), (53, 10800, 7.541), (53, 14397, 7.541), (53, 4186, 7.541), (53, 14741, 7.541), (53, 18671, 7.541), (53, 2167, 7.541), (53, 11703, 7.541), (53, 12549, 7.541), (53, 16592, 7.541), (53, 35849, 7.541), (53, 15583, 7.541), (53, 28223, 7.541), (53, 8086, 7.541), (53, 9330, 7.541), (53, 28121, 7.541), (53, 2924, 7.541), (53, 1250, 7.541), (53, 2759, 7.541), (53, 3784, 7.541), (53, 3785, 7.541), (53, 8536, 7.541), (53, 18507, 7.541), (53, 9367, 7.541), (53, 5114, 7.541), (53, 908, 7.541), (53, 430, 7.541), (53, 123, 7.541), (53, 4725, 7.541), (53, 25183, 7.541), (53, 17895, 7.541), (53, 8425, 7.541), (53, 10793, 7.541), (53, 134, 7.541), (53, 13161, 7.541), (53, 31859, 7.541), (53, 20583, 7.541), (53, 32935, 7.541), (53, 28891, 7.541), (53, 587, 7.541), (53, 15225,

7.541), (53, 8630, 7.541), (53, 8074, 7.541), (53, 9515, 7.541), (53, 934, 7.541), (53, 1889, 7.541), (53, 9041, 7.541), (53, 3731, 7.541), (53, 228, 7.541), (53, 3092, 7.541), (53, 4814, 7.541), (53, 14467, 7.541), (53, 5680, 7.541), (53, 6862, 7.541), (53, 7054, 7.541), (53, 813, 7.541), (53, 467, 7.541), (53, 379, 7.541), (53, 790, 7.541), (53, 10087, 7.541), (53, 11741, 7.541), (53, 395, 7.541), (53, 578, 7.541), (53, 14719, 7.541), (53, 4282, 7.541), (53, 572, 7.541), (53, 34599, 7.541), (53, 512, 7.541), (53, 3655, 7.541), (53, 4437, 7.541), (53, 594, 7.541), (53, 4477, 7.541), (53, 29803, 7.541), (53, 528, 7.541), (53, 166, 7.541), (53, 9253, 7.541), (53, 16524, 7.541), (53, 11757, 7.541), (53, 6702, 7.541), (53, 1482, 7.541), (53, 28171, 7.541), (53, 1292, 7.541), (53, 7647, 7.541), (53, 9513, 7.541), (53, 1498, 7.541), (53, 4901, 7.541), (53, 11285, 7.541), (53, 1686, 7.541), (53, 2889, 7.541), (53, 762, 7.541), (53, 834, 7.541), (53, 11633, 7.541), (53, 405, 7.541), (53, 11241, 7.541), (53, 4981, 7.541), (53, 8915, 7.541), (53, 4182, 7.541), (53, 7338, 7.541), (53, 6573, 7.541), (53, 6880, 7.541), (53, 10155, 7.541), (53, 223, 7.541), (53, 225, 7.541), (53, 6033, 7.541), (53, 22297, 7.541), (53, 28701, 7.541), (53, 4192, 7.541), (53, 11617, 7.541), (53, 8277, 7.541), (53, 7088, 7.541), (53, 468, 7.541), (53, 18679, 7.541), (53, 22535, 7.541), (53, 801, 7.541), (53, 237, 7.541), (53, 4898, 7.541), (53, 6707, 7.541), (53, 18153, 7.541), (53, 17265, 7.541), (53, 23321, 7.541), (53, 18115, 7.541), (53, 14513, 7.541), (53, 20785, 7.541), (53, 23755, 7.541), (53, 442, 7.541), (53, 19815, 7.541), (53, 20507, 7.541), (53, 13601, 7.541), (53, 23281, 7.541), (53, 2993, 7.541), (53, 4214, 7.541), (53, 4063, 7.541), (53, 5355, 7.541), (53, 154, 7.541), (53, 3588, 7.541), (53, 2418, 7.541), (53, 21881, 7.541), (53, 9941, 7.541), (53, 6213, 7.541), (53, 4654, 7.541), (53, 8937, 7.541), (53, 22319, 7.541), (53, 27899, 7.541), (53, 2787, 7.541), (53, 530, 7.541), (53, 740, 7.541), (53, 14713, 7.541), (53, 3503, 7.541), (53, 6045, 7.541), (53, 9656, 7.541), (53, 355, 7.541), (53, 4722, 7.541), (53, 534, 7.541), (53, 3470, 7.541), (53, 513, 7.541), (53, 14227, 7.541), (53, 3457, 7.541), (53, 4752, 7.541), (53, 4181, 7.541), (53, 10620, 7.541), (53, 2923, 7.541), (53, 713, 7.541), (53, 357, 7.541), (53, 6351, 7.541), (53, 4059, 7.541), (53, 376, 7.541), (53, 3958, 7.541), (53, 1004, 7.541), (53, 1887, 7.541), (53, 4472, 7.541), (53, 2963, 7.541), (53, 527, 7.541), (53, 1117, 7.541), (53, 1118, 7.541), (53, 846, 7.541), (53, 1965, 7.541), (53, 4382, 7.541), (53, 1195, 7.541), (53, 232, 7.541), (53, 94, 7.541), (53, 79, 7.541), (53, 1840, 7.541), (53, 481, 7.541), (53, 9062, 7.541), (53, 6746, 7.541), (53, 9314, 7.541), (53, 6594, 7.541), (53, 8129, 7.541), (53, 9756, 7.541), (53, 7769, 7.541), (53, 10067, 7.541), (53, 5081, 7.541), (53, 4835, 7.541), (53, 4177, 7.541), (53, 2164, 7.541), (53, 5177, 7.541), (53, 1726, 7.541), (53, 552, 7.541), (53, 874, 7.541), (53, 3230, 7.541), (53, 408, 7.541), (53, 387, 7.541), (53, 6895, 7.541), (53, 452, 7.541), (53, 450, 7.541), (53, 451, 7.541), (53, 7791, 7.541), (53, 2593, 7.541), (53, 3782, 7.541), (53, 3783, 7.541), (53, 4280, 7.541), (53, 5204, 7.541), (53, 5205, 7.541), (53, 1691, 7.541), (53, 6205, 7.541), (53, 1579, 7.541), (53, 3572, 7.541), (53, 98, 7.541), (53,

3342, 7.541), (53, 4081, 7.541), (53, 6512, 7.541), (53, 4155, 7.541), (53, 5341, 7.541), (53, 6007, 7.541), (53, 8769, 7.541), (53, 437, 7.541), (53, 65, 7.541), (53, 339, 7.541), (53, 5681, 7.541), (53, 7311, 7.541), (53, 6211, 7.541), (53, 4224, 7.541), (53, 27, 7.541), (53, 2129, 7.541), (53, 969, 7.541), (53, 6956, 7.541), (53, 861, 7.541), (53, 3712, 7.541), (53, 5300, 7.541), (53, 4975, 7.541), (53, 6747, 7.541), (53, 2581, 7.541), (53, 5530, 7.541), (53, 5682, 7.541), (53, 6500, 7.541), (53, 3091, 7.541), (53, 67, 7.541), (53, 149, 7.541), (53, 5909, 7.541), (53, 371, 7.541), (53, 372, 7.541), (53, 1313, 7.541), (53, 2961, 7.541), (53, 1132, 7.541), (66, 21, 7.541), (66, 47, 7.541), (66, 591, 7.541), (66, 68, 7.541), (66, 889, 7.541), (66, 1519, 7.541), (66, 60, 7.541), (66, 1, 7.541), (66, 5, 7.541), (66, 61, 7.541), (66, 2025, 7.541), (66, 1361, 7.541), (66, 227, 7.541), (66, 120, 7.541), (66, 1015, 7.541), (66, 2890, 7.541), (66, 239, 7.541), (66, 384, 7.541), (66, 240, 7.541), (66, 2508, 7.541), (66, 1571, 7.541), (66, 241, 7.541), (66, 268, 7.541), (66, 245, 7.541), (66, 133, 7.541), (66, 267, 7.541), (66, 16, 7.541), (66, 263, 7.541), (66, 5258, 7.541), (66, 15809, 7.541), (66, 270, 7.541), (66, 5630, 7.541), (66, 135, 7.541), (66, 256, 7.541), (66, 431, 7.541), (66, 136, 7.541), (66, 11061, 7.541), (66, 138, 7.541), (66, 139, 7.541), (66, 137, 7.541), (66, 248, 7.541), (66, 257, 7.541), (66, 185, 7.541), (66, 249, 7.541), (66, 33, 7.541), (66, 93, 7.541), (66, 147, 7.541), (66, 32281, 7.541), (66, 486, 7.541), (66, 412, 7.541), (66, 43, 7.541), (66, 433, 7.541), (66, 416, 7.541), (66, 97, 7.541), (66, 189, 7.541), (66, 190, 7.541), (66, 191, 7.541), (66, 192, 7.541), (66, 330, 7.541), (66, 585, 7.541), (66, 164, 7.541), (66, 457, 7.541), (66, 877, 7.541), (66, 20, 7.541), (66, 936, 7.541), (66, 597, 7.541), (66, 30, 7.541), (66, 31, 7.541), (66, 32, 7.541), (66, 1210, 7.541), (66, 30276, 7.541), (66, 195, 7.541), (66, 196, 7.541), (66, 12355, 7.541), (66, 2966, 7.541), (66, 853, 7.541), (66, 1943, 7.541), (66, 322, 7.541), (66, 210, 7.541), (66, 710, 7.541), (66, 45, 7.541), (66, 401, 7.541), (66, 44, 7.541), (66, 205, 7.541), (66, 2476, 7.541), (66, 24, 7.541), (66, 400, 7.541), (66, 1827, 7.541), (66, 199, 7.541), (66, 2104, 7.541), (66, 1559, 7.541), (66, 90, 7.541), (66, 16498, 7.541), (66, 25777, 7.541), (66, 169, 7.541), (66, 849, 7.541), (66, 2001, 7.541), (66, 174, 7.541), (66, 182, 7.541), (66, 2236, 7.541), (66, 523, 7.541), (66, 6, 7.541), (66, 10162, 7.541), (66, 7, 7.541), (66, 202, 7.541), (66, 1562, 7.541), (66, 392, 7.541), (66, 15, 7.541), (66, 2605, 7.541), (66, 22, 7.541), (66, 857, 7.541), (66, 66, 7.541), (66, 2904, 7.541), (66, 235, 7.541), (66, 3002, 7.541), (66, 777, 7.541), (66, 1530, 7.541), (66, 145, 7.541), (66, 1604, 7.541), (66, 323, 7.541), (66, 5525, 7.541), (66, 6682, 7.541), (66, 11759, 7.541), (66, 22199, 7.541), (66, 4744, 7.541), (66, 25013, 7.541), (66, 22729, 7.541), (66, 27655, 7.541), (66, 6547, 7.541), (66, 9989, 7.541), (66, 11433, 7.541), (66, 11111, 7.541), (66, 24833, 7.541), (66, 21995, 7.541), (66, 9919, 7.541), (66, 12967, 7.541), (66, 477, 7.541), (66, 9736, 7.541), (66, 5342, 7.541), (66, 2251, 7.541), (66, 3901, 7.541), (66, 6347, 7.541), (66, 7674, 7.541), (66, 10030, 7.541), (66, 2986, 7.541), (66, 22789, 7.541), (66, 10396,

7.541), (66, 13535, 7.541), (66, 20787, 7.541), (66, 7059, 7.541), (66, 10490, 7.541), (66, 31043, 7.541), (66, 31964, 7.541), (66, 33486, 7.541), (66, 36456, 7.541), (66, 10719, 7.541), (66, 14967, 7.541), (66, 6772, 7.541), (66, 8514, 7.541), (66, 14345, 7.541), (66, 31478, 7.541), (66, 1536, 7.541), (66, 10163, 7.541), (66, 12293, 7.541), (66, 5356, 7.541), (66, 10578, 7.541), (66, 28999, 7.541), (66, 10800, 7.541), (66, 14397, 7.541), (66, 4186, 7.541), (66, 14741, 7.541), (66, 18671, 7.541), (66, 11703, 7.541), (66, 12549, 7.541), (66, 16592, 7.541), (66, 35849, 7.541), (66, 15583, 7.541), (66, 28223, 7.541), (66, 8086, 7.541), (66, 9330, 7.541), (66, 28121, 7.541), (66, 2924, 7.541), (66, 1250, 7.541), (66, 2759, 7.541), (66, 3784, 7.541), (66, 3785, 7.541), (66, 8536, 7.541), (66, 18507, 7.541), (66, 9367, 7.541), (66, 5114, 7.541), (66, 123, 7.541), (66, 4725, 7.541), (66, 25183, 7.541), (66, 17895, 7.541), (66, 8425, 7.541), (66, 10793, 7.541), (66, 13161, 7.541), (66, 31859, 7.541), (66, 20583, 7.541), (66, 32935, 7.541), (66, 28891, 7.541), (66, 587, 7.541), (66, 15225, 7.541), (66, 8630, 7.541), (66, 8074, 7.541), (66, 9515, 7.541), (66, 1889, 7.541), (66, 9041, 7.541), (66, 3731, 7.541), (66, 228, 7.541), (66, 3092, 7.541), (66, 4814, 7.541), (66, 14467, 7.541), (66, 6862, 7.541), (66, 7054, 7.541), (66, 467, 7.541), (66, 379, 7.541), (66, 790, 7.541), (66, 10087, 7.541), (66, 11741, 7.541), (66, 395, 7.541), (66, 578, 7.541), (66, 14719, 7.541), (66, 4282, 7.541), (66, 572, 7.541), (66, 34599, 7.541), (66, 512, 7.541), (66, 3655, 7.541), (66, 4437, 7.541), (66, 594, 7.541), (66, 4477, 7.541), (66, 29803, 7.541), (66, 528, 7.541), (66, 166, 7.541), (66, 9253, 7.541), (66, 16524, 7.541), (66, 11757, 7.541), (66, 6702, 7.541), (66, 1482, 7.541), (66, 28171, 7.541), (66, 7647, 7.541), (66, 9513, 7.541), (66, 1498, 7.541), (66, 4901, 7.541), (66, 11285, 7.541), (66, 2889, 7.541), (66, 762, 7.541), (66, 834, 7.541), (66, 11633, 7.541), (66, 405, 7.541), (66, 11241, 7.541), (66, 4981, 7.541), (66, 8915, 7.541), (66, 4182, 7.541), (66, 7338, 7.541), (66, 6573, 7.541), (66, 6880, 7.541), (66, 10155, 7.541), (66, 223, 7.541), (66, 225, 7.541), (66, 6033, 7.541), (66, 22297, 7.541), (66, 28701, 7.541), (66, 4192, 7.541), (66, 11617, 7.541), (66, 8277, 7.541), (66, 7088, 7.541), (66, 468, 7.541), (66, 18679, 7.541), (66, 22535, 7.541), (66, 801, 7.541), (66, 237, 7.541), (66, 4898, 7.541), (66, 6707, 7.541), (66, 18153, 7.541), (66, 17265, 7.541), (66, 23321, 7.541), (66, 18115, 7.541), (66, 14513, 7.541), (66, 20785, 7.541), (66, 23755, 7.541), (66, 442, 7.541), (66, 19815, 7.541), (66, 20507, 7.541), (66, 13601, 7.541), (66, 23281, 7.541), (66, 2993, 7.541), (66, 4214, 7.541), (66, 64, 7.541), (66, 4063, 7.541), (66, 5355, 7.541), (66, 154, 7.541), (66, 3588, 7.541), (66, 2418, 7.541), (66, 21881, 7.541), (66, 9941, 7.541), (66, 6213, 7.541), (66, 4654, 7.541), (66, 8937, 7.541), (66, 22319, 7.541), (66, 27899, 7.541), (66, 3167, 7.541), (66, 2787, 7.541), (66, 14713, 7.541), (66, 3503, 7.541), (66, 6045, 7.541), (66, 9656, 7.541), (66, 355, 7.541), (66, 4722, 7.541), (66, 534, 7.541), (66, 3470, 7.541), (66, 513, 7.541), (66, 14227, 7.541), (66, 3457, 7.541), (66, 4752, 7.541), (66, 4181, 7.541), (66, 10620, 7.541), (66, 2923, 7.541), (66, 713, 7.541), (66, 357, 7.541), (66, 6351, 7.541),

```
(66, 4059, 7.541), (66, 3958, 7.541), (66, 1004, 7.541), (66, 1887,
7.541), (66, 4472, 7.541), (66, 2963, 7.541), (66, 527, 7.541), (66,
1117, 7.541), (66, 1118, 7.541), (66, 1965, 7.541), (66, 4382, 7.541),
(66, 1195, 7.541), (66, 94, 7.541), (66, 79, 7.541), (66, 177, 7.541),
(66, 1840, 7.541), (66, 481, 7.541), (66, 9062, 7.541), (66, 6746,
7.541), (66, 9314, 7.541), (66, 6594, 7.541), (66, 8129, 7.541), (66,
9756, 7.541), (66, 7769, 7.541), (66, 10067, 7.541), (66, 5081,
7.541), (66, 4835, 7.541), (66, 4177, 7.541), (66, 2164, 7.541), (66,
5177, 7.541), (66, 1726, 7.541), (66, 3230, 7.541), (66, 408, 7.541),
(66, 387, 7.541), (66, 6895, 7.541), (66, 452, 7.541), (66, 450,
7.541), (66, 451, 7.541), (66, 7791, 7.541), (66, 2593, 7.541), (66,
3782, 7.541), (66, 3783, 7.541), (66, 4280, 7.541), (66, 5204, 7.541),
(66, 5205, 7.541), (66, 1691, 7.541), (66, 6205, 7.541), (66, 1579,
7.541), (66, 3572, 7.541), (66, 98, 7.541), (66, 3342, 7.541), (66,
4081, 7.541), (66, 6512, 7.541), (66, 4155, 7.541), (66, 5341, 7.541),
(66, 6007, 7.541), (66, 8769, 7.541), (66, 437, 7.541), (66, 65,
7.541), (66, 339, 7.541), (66, 5681, 7.541), (66, 7311, 7.541), (66,
6211, 7.541), (66, 4224, 7.541), (66, 27, 7.541), (66, 2129, 7.541),
(66, 969, 7.541), (66, 6956, 7.541), (66, 861, 7.541), (66, 3712,
7.541), (66, 5300, 7.541), (66, 4975, 7.541), (66, 6747, 7.541), (66,
2581, 7.541), (66, 5530, 7.541), (66, 5682, 7.541), (66, 6500, 7.541),
(66, 3091, 7.541), (66, 67, 7.541), (66, 149, 7.541), (66, 5909,
7.541)]
```

Implémentation des métriques

Top N

```python
def top_n(predictions, N, rating_min):
  df = pd.DataFrame(predictions)
  df = df[df["est"] >= rating_min].drop(["r_ui", "details"], axis=1)
  df = df.sort_values(by="est", ascending=False)
  df = df.groupby("uid").head(N)
  df = df.groupby("uid").agg({"iid": list, "est": list})
  df.columns = ["topn", "est"]
  return df

# topn = top_n(predictions, N=100, rating_min=0)
```

Cumulative Hit Rate

Le hit rate correspond au cumulative hit rate avec un rating_min=0.

```python
def count_hit(row):
  if isinstance(row["topn"], list):
    return 1 if row["iid"] in row["topn"] else 0
  else:
    return 0
```

```python
def cumulative_hit_rate(topn, test):
  df_test = pd.DataFrame(test, columns = ["uid", "iid", "r_ui"])
  df_test =  df_test.set_index("uid")
  df_test["topn"] = topn["topn"]


  # Calcul du nombre de hits:
  hits = df_test.apply(count_hit, axis=1).sum()
  res = hits/len(test)

  return res

# cumulative_hit_rate(topn, test)
```

Average Reciprocal Hit Rate

```python
def get_weight(row):
  """
  Si iid est dans le top N, on renvoie 1/p où p est la position de iid
dans le top N
  Sinon, on renvoie 0
  """
  if isinstance(row["topn"], list):
    if row["iid"] in row["topn"]:
      return 1/(row["topn"].index(row["iid"]) + 1)
    else:
      return 0
  else:
    return 0

def average_reciprocal_hit_rate(topn, test):
  df_test = pd.DataFrame(test, columns = ["uid", "iid", "r_ui"])
  df_test =  df_test.set_index("uid")
  df_test["topn"] = topn["topn"]
  weights = df_test.apply(get_weight, axis=1).sum()
  res = weights/len(test)

  return res

# average_reciprocal_hit_rate(topn, test)
```

User Coverage

```python
def check_threshold(row, threshold):
  """
  Retourne 1 s'il existe une note estimée au dessus du seuil, 0 sinon
  """
  if isinstance(row["est"], list):
    return 1 if row["est"][0] >= threshold else 0
```

```
  else:
    return 0

def user_coverage(topn, threshold=0):
  coverage = topn.apply(check_threshold, threshold=threshold,
axis=1).sum()
  res = coverage/len(test)

  return res

# user_coverage(topn, threshold=5)
```

Création d'une fonction d'évaluation des modèles

```
import json
performances = {}

def evaluate_model(model, name, test, antitest=None, N=10,
rating_min=0, threshold=0):
  predictions = model.test(test)
  mae = accuracy.mae(predictions, verbose=False)
  rmse = accuracy.rmse(predictions, verbose=False)
  performances[name] = {"MAE": mae, "RMSE": rmse}

  if antitest is not None:
    predictions = model.test(antitest)
    topn = top_n(N=N, predictions=predictions, rating_min=rating_min)
    chr = cumulative_hit_rate(topn=topn, test=test)
    arhr = average_reciprocal_hit_rate(topn=topn, test=test)
    uc = user_coverage(topn, threshold=threshold)

    performances[name]["Cumulative Hit Rate"] = chr
    performances[name]["Average Reciprocal Hit Rate"] = arhr
    performances[name]["User Coverage"] = uc

  print(json.dumps(performances, indent=2))

# evaluate_model(model=random_model, name="Modele gaussien",
test=test, antitest=antitest, threshold=4, N=50, rating_min=2.5)
```

Collaborative Filtering

KNN basique (KNNBasic)

```
hyperparameters_knn_basic = {
    "k" : [20, 30, 40],
    "min_k" : [1, 3, 5],
    "sim_options": {
        "name": ["cosine", "msd", "pearson"],
        "user_based": [True]
```

```
    }
}

gs_knn_basic = GridSearchCV(
  KNNBasic,
  param_grid=hyperparameters_knn_basic,
  cv = 2,
  measures=['mae'],
)

gs_knn_basic.fit(data)
```

```
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
```

```
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
```

```
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
```

```
best_knn_basic = gs_knn_basic.best_estimator["mae"]
print(best_knn_basic)
```

```
<surprise.prediction_algorithms.knns.KNNBasic object at
0x000001D82A344200>
```

```
best_knn_basic.fit(train)
```

```
Computing the cosine similarity matrix...
Done computing similarity matrix.
```

```
<surprise.prediction_algorithms.knns.KNNBasic at 0x1d82a344200>
```

```
best_knn_basic.test(test)
```

```
[Prediction(uid=23, iid=13161, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=48, iid=97, r_ui=9.0, est=8.00547761883207,
details={'actual_k': 3, 'was_impossible': False}),
 Prediction(uid=4, iid=123, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=1, iid=12355, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=47, iid=4654, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
```

```
 Prediction(uid=23, iid=2904, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=20, iid=263, r_ui=9.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=48, iid=1535, r_ui=7.0, est=8.002006622039215,
details={'actual_k': 3, 'was_impossible': False}),
 Prediction(uid=47, iid=66, r_ui=9.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=48, iid=4725, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=1, iid=120, r_ui=9.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=23, iid=8937, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=4, iid=11433, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=1, iid=7, r_ui=9.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=4, iid=240, r_ui=9.0, est=7.685871796495967,
details={'actual_k': 3, 'was_impossible': False}),
 Prediction(uid=23, iid=4192, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=48, iid=857, r_ui=5.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=37, iid=60, r_ui=10.0, est=8.0, details={'actual_k':
4, 'was_impossible': False}),
 Prediction(uid=4, iid=2167, r_ui=10.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=1, iid=2605, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=4, iid=15583, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=66, iid=1698, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=23, iid=64, r_ui=6.0, est=7.011908271121798,
details={'actual_k': 3, 'was_impossible': False}),
 Prediction(uid=1, iid=317, r_ui=7.0, est=8.24628899521895,
details={'actual_k': 4, 'was_impossible': False}),
 Prediction(uid=47, iid=245, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=9, iid=93, r_ui=10.0, est=7.359435836627821,
details={'actual_k': 3, 'was_impossible': False}),
 Prediction(uid=20, iid=512, r_ui=9.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
```

```
 Prediction(uid=4, iid=13161, r_ui=5.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=48, iid=205, r_ui=8.0, est=8.337354527560068,
details={'actual_k': 6, 'was_impossible': False}),
 Prediction(uid=1, iid=392, r_ui=8.0, est=6.649200580264849,
details={'actual_k': 3, 'was_impossible': False}),
 Prediction(uid=1, iid=1210, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=4, iid=4814, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=48, iid=157, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=1, iid=857, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=23, iid=17265, r_ui=9.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=47, iid=190, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=4, iid=2986, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=20, iid=317, r_ui=8.0, est=8.236608983808132,
details={'actual_k': 4, 'was_impossible': False}),
 Prediction(uid=48, iid=9062, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=48, iid=1210, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=1, iid=33, r_ui=9.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=4, iid=12549, r_ui=5.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=48, iid=9367, r_ui=5.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=4, iid=10490, r_ui=5.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=48, iid=853, r_ui=8.0, est=7.666680957082636,
details={'actual_k': 3, 'was_impossible': False}),
 Prediction(uid=4, iid=6772, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=48, iid=5205, r_ui=9.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
```

```
 Prediction(uid=20, iid=594, r_ui=5.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=66, iid=101, r_ui=10.0, est=8.676049812202265,
details={'actual_k': 3, 'was_impossible': False}),
 Prediction(uid=1, iid=245, r_ui=9.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=20, iid=433, r_ui=9.0, est=8.338579585213871,
details={'actual_k': 3, 'was_impossible': False}),
 Prediction(uid=47, iid=121, r_ui=10.0, est=9.001836315779556,
details={'actual_k': 5, 'was_impossible': False}),
 Prediction(uid=20, iid=9253, r_ui=10.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=1, iid=1519, r_ui=7.0, est=9.000000000000002,
details={'actual_k': 3, 'was_impossible': False}),
 Prediction(uid=47, iid=587, r_ui=8.0, est=6.338329699327272,
details={'actual_k': 3, 'was_impossible': False}),
 Prediction(uid=4, iid=908, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=9, iid=379, r_ui=1.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=23, iid=1535, r_ui=8.0, est=8.001455074950647,
details={'actual_k': 3, 'was_impossible': False}),
 Prediction(uid=47, iid=4382, r_ui=5.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=23, iid=245, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=1, iid=30276, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=47, iid=317, r_ui=7.0, est=8.248919427093801,
details={'actual_k': 4, 'was_impossible': False}),
 Prediction(uid=23, iid=18153, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=48, iid=166, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=37, iid=195, r_ui=9.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=48, iid=2904, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=48, iid=27, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=23, iid=121, r_ui=9.0, est=9.016150485380466,
details={'actual_k': 5, 'was_impossible': False}),
 Prediction(uid=48, iid=450, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=20, iid=1698, r_ui=9.0, est=7.55375,
```

```
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=20, iid=2508, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=47, iid=846, r_ui=9.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=48, iid=2593, r_ui=9.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=37, iid=1698, r_ui=9.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=4, iid=18507, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=1, iid=121, r_ui=9.0, est=9.005222540805791,
details={'actual_k': 5, 'was_impossible': False}),
 Prediction(uid=66, iid=134, r_ui=8.0, est=6.994881411356107,
details={'actual_k': 4, 'was_impossible': False}),
 Prediction(uid=47, iid=710, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=4, iid=10800, r_ui=9.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=4, iid=298, r_ui=5.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=9, iid=174, r_ui=8.0, est=6.330904597125643,
details={'actual_k': 3, 'was_impossible': False}),
 Prediction(uid=4, iid=14741, r_ui=9.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=20, iid=430, r_ui=9.0, est=7.504690563124911,
details={'actual_k': 6, 'was_impossible': False}),
 Prediction(uid=23, iid=43, r_ui=9.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=47, iid=2167, r_ui=9.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=47, iid=3958, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=48, iid=801, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=1, iid=164, r_ui=9.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=48, iid=530, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=1, iid=93, r_ui=8.0, est=7.329542868137943,
details={'actual_k': 3, 'was_impossible': False}),
 Prediction(uid=48, iid=834, r_ui=4.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=23, iid=5355, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
```

```
unknown.'}),
 Prediction(uid=48, iid=9041, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=48, iid=2025, r_ui=6.0, est=8.248414105529667,
details={'actual_k': 4, 'was_impossible': False}),
 Prediction(uid=23, iid=270, r_ui=10.0, est=7.975084606793245,
details={'actual_k': 5, 'was_impossible': False}),
 Prediction(uid=4, iid=8074, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=23, iid=11633, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=53, iid=256, r_ui=10.0, est=8.316148549539495,
details={'actual_k': 3, 'was_impossible': False}),
 Prediction(uid=1, iid=1604, r_ui=5.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=23, iid=4654, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=23, iid=237, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=1, iid=137, r_ui=9.0, est=8.676896461822656,
details={'actual_k': 3, 'was_impossible': False}),
 Prediction(uid=20, iid=457, r_ui=10.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=48, iid=32, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=47, iid=2963, r_ui=9.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=48, iid=405, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=4, iid=31043, r_ui=9.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=48, iid=5355, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=47, iid=1117, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=37, iid=534, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=1, iid=1222, r_ui=7.0, est=8.346685780398936,
details={'actual_k': 3, 'was_impossible': False}),
 Prediction(uid=4, iid=3901, r_ui=10.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=48, iid=9756, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
```

```
unknown.'}),
 Prediction(uid=4, iid=6547, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=48, iid=30, r_ui=6.0, est=7.016955482124022,
details={'actual_k': 3, 'was_impossible': False}),
 Prediction(uid=23, iid=5525, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=48, iid=6351, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=48, iid=5342, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=66, iid=846, r_ui=9.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=23, iid=226, r_ui=9.0, est=8.11057652914791,
details={'actual_k': 8, 'was_impossible': False}),
 Prediction(uid=20, iid=11061, r_ui=10.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=4, iid=9330, r_ui=5.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=48, iid=10067, r_ui=5.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=53, iid=330, r_ui=10.0, est=7.669833171811461,
details={'actual_k': 3, 'was_impossible': False}),
 Prediction(uid=66, iid=1575, r_ui=9.0, est=7.8017781440623,
details={'actual_k': 5, 'was_impossible': False}),
 Prediction(uid=48, iid=5300, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=48, iid=467, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=23, iid=30, r_ui=1.0, est=7.073013961188089,
details={'actual_k': 3, 'was_impossible': False}),
 Prediction(uid=4, iid=1575, r_ui=8.0, est=7.80027692798712,
details={'actual_k': 5, 'was_impossible': False}),
 Prediction(uid=48, iid=7, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=9, iid=59, r_ui=8.0, est=7.80248729095343,
details={'actual_k': 5, 'was_impossible': False}),
 Prediction(uid=23, iid=7088, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=47, iid=71, r_ui=7.0, est=8.145536732301093,
details={'actual_k': 7, 'was_impossible': False}),
 Prediction(uid=48, iid=123, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=20, iid=1535, r_ui=9.0, est=7.998581071279167,
```

```
details={'actual_k': 3, 'was_impossible': False}),
 Prediction(uid=37, iid=2787, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=47, iid=527, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=66, iid=122, r_ui=9.0, est=7.012348297208071,
details={'actual_k': 3, 'was_impossible': False}),
 Prediction(uid=48, iid=5081, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=1, iid=248, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=20, iid=3002, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=4, iid=3092, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=1, iid=1530, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=1, iid=1689, r_ui=8.0, est=9.003760922771484,
details={'actual_k': 4, 'was_impossible': False}),
 Prediction(uid=4, iid=150, r_ui=9.0, est=8.204783077907718,
details={'actual_k': 5, 'was_impossible': False}),
 Prediction(uid=9, iid=101, r_ui=9.0, est=8.678313112357545,
details={'actual_k': 3, 'was_impossible': False}),
 Prediction(uid=53, iid=2986, r_ui=9.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=48, iid=5630, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=66, iid=1818, r_ui=8.0, est=7.798908641416818,
details={'actual_k': 5, 'was_impossible': False}),
 Prediction(uid=48, iid=387, r_ui=9.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=23, iid=6, r_ui=8.0, est=8.680877140088546,
details={'actual_k': 3, 'was_impossible': False}),
 Prediction(uid=4, iid=1536, r_ui=4.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=1, iid=486, r_ui=9.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=48, iid=4654, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=48, iid=597, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=47, iid=2605, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=48, iid=936, r_ui=4.0, est=7.55375,
```

```
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=20, iid=585, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=23, iid=14345, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=20, iid=395, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=48, iid=3230, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=4, iid=20787, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=23, iid=166, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=48, iid=101, r_ui=8.0, est=8.668060383760796,
details={'actual_k': 3, 'was_impossible': False}),
 Prediction(uid=1, iid=147, r_ui=7.0, est=8.503862162941145,
details={'actual_k': 4, 'was_impossible': False}),
 Prediction(uid=23, iid=5, r_ui=8.0, est=7.806632068102641,
details={'actual_k': 5, 'was_impossible': False}),
 Prediction(uid=23, iid=9041, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=23, iid=9941, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=23, iid=27899, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=37, iid=3457, r_ui=10.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=23, iid=20507, r_ui=9.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=48, iid=339, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=48, iid=3783, r_ui=9.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=37, iid=182, r_ui=10.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=48, iid=408, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=48, iid=452, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
```

```
 Prediction(uid=48, iid=2167, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=47, iid=357, r_ui=5.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=4, iid=10030, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=37, iid=330, r_ui=8.0, est=7.66774086631341,
details={'actual_k': 3, 'was_impossible': False}),
 Prediction(uid=53, iid=72, r_ui=10.0, est=8.67448011751532,
details={'actual_k': 6, 'was_impossible': False}),
 Prediction(uid=20, iid=164, r_ui=10.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=47, iid=232, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=48, iid=5681, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=48, iid=164, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=1, iid=1571, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=48, iid=4182, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=20, iid=11741, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=23, iid=190, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=4, iid=10396, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=23, iid=11617, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=23, iid=20787, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=4, iid=317, r_ui=4.0, est=8.24514882902414,
details={'actual_k': 4, 'was_impossible': False}),
 Prediction(uid=48, iid=6956, r_ui=7.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=37, iid=853, r_ui=9.0, est=7.6624551096752676,
details={'actual_k': 3, 'was_impossible': False}),
 Prediction(uid=66, iid=48, r_ui=8.0, est=6.668312818438477,
details={'actual_k': 3, 'was_impossible': False}),
 Prediction(uid=47, iid=401, r_ui=9.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'}),
 Prediction(uid=20, iid=240, r_ui=7.0, est=7.675434456802269,
```

```
details={'actual_k': 3, 'was_impossible': False}),
 Prediction(uid=47, iid=10620, r_ui=6.0, est=7.55375,
details={'was_impossible': True, 'reason': 'User and/or item is
unknown.'}),
 Prediction(uid=1, iid=523, r_ui=8.0, est=7.55375,
details={'was_impossible': True, 'reason': 'Not enough neighbors.'})]
```

```
# On observe beaucoup de "was impossible = True" le modèle n'arrive
pas à faire des prédictions
trainset = best_knn_basic.trainset
antitest = trainset.build_anti_testset()

evaluate_model(model=best_knn_basic, name="KNN Basic", test=test,
antitest=antitest,threshold=4, N=50, rating_min=5)
```


```
{
  "KNN Basic": {
    "MAE": 1.2590639819050304,
    "RMSE": 1.5634364772503673,
    "Cumulative Hit Rate": 0.24,
    "Average Reciprocal Hit Rate": 0.018800024264368897,
    "User Coverage": 0.05
  }
}
```


KNN avec moyennes (KNNWithMeans)

```
hyperparameters_knn_means = {
    "k" : [20, 30, 40],
    "min_k" : [1, 3, 5],
    "sim_options": {
        "name": ["cosine", "msd", "pearson"],
        "user_based": [True]
    }
}

gs_knn_means = GridSearchCV(
  KNNWithMeans,
  param_grid=hyperparameters_knn_means,
  cv = 2,
  measures=['mae'],
)

gs_knn_means.fit(data)
```

```
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
```

```
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
```

```
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
```

```
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
```

```
best_knn_means = gs_knn_means.best_estimator["mae"]

best_knn_means.fit(train)
```

```
Computing the cosine similarity matrix...
Done computing similarity matrix.
```

```
<surprise.prediction_algorithms.knns.KNNWithMeans at 0x1d878e08fe0>
```

```
evaluate_model(model=best_knn_means, name="KNN avec moyennes",
test=test, antitest=antitest,threshold=4, N=50, rating_min=5)
```


```
{
  "KNN Basic": {
    "MAE": 1.2590639819050304,
    "RMSE": 1.5634364772503673,
    "Cumulative Hit Rate": 0.24,
    "Average Reciprocal Hit Rate": 0.018800024264368897,
    "User Coverage": 0.05
  },
  "KNN avec moyennes": {
    "MAE": 1.1631594102502676,
    "RMSE": 1.485557703361385,
    "Cumulative Hit Rate": 0.225,
    "Average Reciprocal Hit Rate": 0.021470230784661065,
    "User Coverage": 0.05
  }
}
```


KNN centré réduit (KNNWithZScore)

```
hyperparameters_knn_zscore = {
    "k" : [20, 30, 40],
    "min_k" : [1, 3, 5],
    "sim_options": {
        "name": ["cosine", "msd", "pearson"],
        "user_based": [True]
    }
}

gs_knn_zscore = GridSearchCV(
  KNNWithZScore,
  param_grid=hyperparameters_knn_zscore,
  cv = 2,
  measures=['mae'],
)
```

```
gs_knn_zscore.fit(data)
```

```
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
```

```
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
```

```
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
```

```
best_knn_zscore = gs_knn_zscore.best_estimator["mae"]
```

```
best_knn_zscore.fit(train)
```

```
Computing the msd similarity matrix...
Done computing similarity matrix.
```

```
<surprise.prediction_algorithms.knns.KNNWithZScore at 0x1d810680aa0>
```

```
evaluate_model(model=best_knn_zscore, name="KNN centré réduit",
test=test, antitest=antitest, threshold=4, N=50, rating_min=5)
```


```
{
  "KNN Basic": {
    "MAE": 1.2590639819050304,
    "RMSE": 1.5634364772503673,
    "Cumulative Hit Rate": 0.24,
    "Average Reciprocal Hit Rate": 0.018800024264368897,
    "User Coverage": 0.05
  },
  "KNN avec moyennes": {
    "MAE": 1.1631594102502676,
    "RMSE": 1.485557703361385,
    "Cumulative Hit Rate": 0.225,
    "Average Reciprocal Hit Rate": 0.021470230784661065,
    "User Coverage": 0.05
  },
  "KNN centr\u00e9 r\u00e9duit": {
    "MAE": 1.1887512321191276,
    "RMSE": 1.5267650726402253,
    "Cumulative Hit Rate": 0.245,
    "Average Reciprocal Hit Rate": 0.02455819388364281,
    "User Coverage": 0.05
  }
}
```

KNN en tenant compte des biais b_u et b_i (KNNBaseline)

```
hyperparameters_knn_bu_bi = {
    "k" : [20, 30, 40],
    "min_k" : [1, 3, 5],
    "sim_options": {
        "name": ["cosine", "msd", "pearson"],
        "user_based": [True]
    },
    "bsl_options": {
        "method": ["sgd"],
        "learning_rate": [0.005, 0.01]
    }
}

gs_knn_bu_bi = GridSearchCV(
  KNNBaseline,
  param_grid=hyperparameters_knn_bu_bi,
  cv = 2,
  measures=['mae'],
)

gs_knn_bu_bi.fit(data)
```

```
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
```

```
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
```

```
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
```

```
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
```

```
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
```

```
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
```

```
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the cosine similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
Computing the pearson similarity matrix...
Done computing similarity matrix.
Estimating biases using sgd...
```

```
Computing the pearson similarity matrix...
Done computing similarity matrix.
```

```
best_knn_bu_bi = gs_knn_bu_bi.best_estimator["mae"]
```

```
best_knn_bu_bi.fit(train)
```

```
Estimating biases using sgd...
Computing the msd similarity matrix...
Done computing similarity matrix.
```

```
<surprise.prediction_algorithms.knns.KNNBaseline at 0x1d82a344fb0>
```

```
evaluate_model(model=best_knn_bu_bi, name="KNN avec les biais bu et
bi", test=test, antitest=antitest, threshold=4, N=50, rating_min=5)
```


```
{
  "KNN Basic": {
    "MAE": 1.2590639819050304,
    "RMSE": 1.5634364772503673,
    "Cumulative Hit Rate": 0.24,
    "Average Reciprocal Hit Rate": 0.018800024264368897,
    "User Coverage": 0.05
  },
  "KNN avec moyennes": {
    "MAE": 1.1631594102502676,
    "RMSE": 1.485557703361385,
    "Cumulative Hit Rate": 0.225,
    "Average Reciprocal Hit Rate": 0.021470230784661065,
    "User Coverage": 0.05
  },
  "KNN centr\u00e9 r\u00e9duit": {
    "MAE": 1.1887512321191276,
    "RMSE": 1.5267650726402253,
    "Cumulative Hit Rate": 0.245,
    "Average Reciprocal Hit Rate": 0.02455819388364281,
    "User Coverage": 0.05
  },
  "KNN avec les biais bu et bi": {
    "MAE": 1.082168948536124,
    "RMSE": 1.4308496700781395,
    "Cumulative Hit Rate": 0.225,
    "Average Reciprocal Hit Rate": 0.032892209021112395,
    "User Coverage": 0.05
  }
}
```

1. Factorisation de Matrice sans Feedbacks Implicites : SVD (Singular Value Decomposition)

La méthode SVD est l'une des plus populaires pour la factorisation de matrice. Elle ne tient pas compte des feedbacks implicites et se base uniquement sur les ratings explicites des utilisateurs.

Étape 1 : Définir un dictionnaire d'hyperparamètres pour SVD

Les hyperparamètres principaux pour SVD incluent :

n_factors : Le nombre de facteurs latents.

n_epochs : Le nombre d'itérations pour entraîner le modèle.

lr_all : Le taux d'apprentissage pour la descente de gradient.

reg_all : Le coefficient de régularisation.

```
param_grid = {
    'n_factors': [50, 100, 150],   # Nombre de facteurs latents
    'n_epochs': [20, 30, 40],      # Nombre d'itérations
    'lr_all': [0.002, 0.005],      # Taux d'apprentissage
    'reg_all': [0.02, 0.1]         # Régularisation
}

# Initialiser GridSearchCV pour le modèle SVD
gs = GridSearchCV(SVD, param_grid, measures=['rmse', 'mae'], cv=5)

# Appliquer la validation croisée sur le dataset
gs.fit(data)

# Afficher les meilleurs hyperparamètres
print(gs.best_params['rmse'])
```

```
{'n_factors': 100, 'n_epochs': 40, 'lr_all': 0.005, 'reg_all': 0.02}
```

```
# Récupérer les meilleurs hyperparamètres
best_params = gs.best_params['rmse']

# Entraîner le modèle avec ces hyperparamètres
SVDModel = SVD(n_factors=best_params['n_factors'],
           n_epochs=best_params['n_epochs'],
           lr_all=best_params['lr_all'],
           reg_all=best_params['reg_all'])

# Entraîner le modèle sur le jeu d'entraînement
SVDModel.fit(train)

# Tester le modèle sur le jeu de test
predictions = SVDModel.test(test)

# évaluer le modèle (Calculer et afficher les métriques)
# Calculer les métriques
```

```
rmse = accuracy.rmse(predictions, verbose=True)
mae = accuracy.mae(predictions, verbose=True)
```

```
RMSE: 1.4727
MAE:  1.1054
```

2. Factorisation de Matrice avec Feedbacks Implicites : SVD++

Le modèle SVD++ prend en compte à la fois les feedbacks explicites et implicites. Cela inclut des interactions comme des clics, des ajouts à une liste de souhaits, des vues de films, etc., en plus des évaluations explicites données par les utilisateurs.

Étape 1 : Définir un dictionnaire d'hyperparamètres pour SVD++

Les hyperparamètres de SVD++ sont similaires à ceux de SVD, avec des facteurs supplémentaires pour les feedbacks implicites :

Étape 2 : Validation croisée avec GridSearchCV pour SVD++

La validation croisée se fait de la même manière que pour SVD, mais avec le modèle SVD++ :

Étape 3 : Mesurer l'efficacité sur le jeu de test (MAE et RMSE)

Vous pouvez maintenant entraîner le modèle SVD++ avec les meilleurs hyperparamètres et mesurer son efficacité :

```
# Initialiser GridSearchCV pour le modèle SVD++
gs = GridSearchCV(SVDpp, param_grid, measures=['rmse', 'mae'], cv=5)

# Appliquer la validation croisée
gs.fit(data)

# Afficher les meilleurs hyperparamètres
print(gs.best_params['rmse'])
```

```
{'n_factors': 100, 'n_epochs': 30, 'lr_all': 0.005, 'reg_all': 0.1}
```

```
# Récupérer les meilleurs hyperparamètres
best_params = gs.best_params['rmse']

# Entraîner le modèle avec ces hyperparamètres
SVDPPModel = SVDpp(n_factors=best_params['n_factors'],
             n_epochs=best_params['n_epochs'],
             lr_all=best_params['lr_all'],
             reg_all=best_params['reg_all'])

# Entraîner et tester le modèle
SVDPPModel.fit(train)
predictions = SVDPPModel.test(test)

# Calculer les métriques
rmse = accuracy.rmse(predictions, verbose=True)
mae = accuracy.mae(predictions, verbose=True)
```

```
RMSE: 1.4436
MAE:  1.0919
```

One Slope

```
# Créer et entraîner le modèle Slope One
SlopeOneModel = SlopeOne()
SlopeOneModel.fit(train)
```

```
<surprise.prediction_algorithms.slope_one.SlopeOne at 0x1d879b2baa0>
```

```
# Faire des prédictions sur le jeu de test
predictions = SlopeOneModel.test(test)

# Calculer le MAE et le RMSE pour évaluer les performances du modèle
mae = accuracy.mae(predictions)
rmse = accuracy.rmse(predictions)
```

```
MAE:  1.2049
RMSE: 1.5952
```

Co Clustering

```
# Définir un dictionnaire d'hyperparamètres à explorer
hyperparameters = {
    'n_cltr_u': [3, 5, 7],  # Nombre de clusters d'utilisateurs
    'n_cltr_i': [3, 5, 7],  # Nombre de clusters d'items
    'n_epochs': [10, 20, 30]  # Nombre d'itérations pour
l'entraînement
}

# Appliquer GridSearchCV pour trouver les meilleurs hyperparamètres
gs = GridSearchCV(CoClustering, hyperparameters, measures=['rmse',
'mae'], cv=2)
gs.fit(data)

# Afficher les meilleurs hyperparamètres et les résultats associés
print("Meilleurs hyperparamètres:", gs.best_params['rmse'])
print("Meilleur score RMSE:", gs.best_score['rmse'])
```

```
Meilleurs hyperparamètres: {'n_cltr_u': 3, 'n_cltr_i': 7, 'n_epochs':
20}
Meilleur score RMSE: 1.5988770383292428
```

```
# Entraîner le modèle avec les meilleurs hyperparamètres
best_algo = gs.best_estimator['rmse']
best_algo.fit(train)
```

```
<surprise.prediction_algorithms.co_clustering.CoClustering at
0x1d82a347320>
```

```
# Faire des prédictions sur le jeu de test
predictions = best_algo.test(test)

# Mesurer le MAE et le RMSE
mae = accuracy.mae(predictions)
rmse = accuracy.rmse(predictions)
```

```
MAE:  1.2773
RMSE: 1.6214
```

Implémenter une fonction prenant un identifiant utilisateur en entrée et qui retournera le topN pour cet utilisateur sous la forme "titre" - "note moyenne" - "nombre de notes".

```
# 1. Calculer le nombre de notes par anime
nb_ratings_per_anime =
filtered_scores.groupby(by="anime_id").size().rename("nb_ratings")

# 2. Joindre le nombre de notes au dataset principal
# En utilisant merge pour ajouter la colonne nb_ratings au dataset
score_dataset
score_dataset_with_ratings =
filtered_scores.merge(nb_ratings_per_anime, on="anime_id", how="left")

# Optionnel : Vérifier le résultat
print(score_dataset_with_ratings.head())
```

```
   user_id Username  anime_id     Anime Title  rating  nb_ratings
0        1    Xinil        21       One Piece       9           3
1        1    Xinil        48     .hack//Sign       7           4
2        1    Xinil        53   Ai Yori Aoshi       7           4
3        1    Xinil        47           Akira       5           2
4        1    Xinil       591  Amaenaide yo!!       6           2
```

```
# 1. Calculer la note moyenne par anime
average_ratings_per_anime =
score_dataset_with_ratings.groupby(by="anime_id")
["rating"].mean().rename("average_ratings")

# 2. Joindre la note moyenne au dataset principal
# En utilisant merge pour ajouter la colonne average_ratings au
dataset score_dataset
score_dataset_with_avg_ratings =
score_dataset_with_ratings.merge(average_ratings_per_anime,
on="anime_id", how="left")

# Optionnel : Vérifier le résultat
print(score_dataset_with_avg_ratings.head())
```

```
   user_id Username  anime_id     Anime Title  rating  nb_ratings  \
0        1    Xinil        21       One Piece       9           3
1        1    Xinil        48     .hack//Sign       7           4
2        1    Xinil        53   Ai Yori Aoshi       7           4
3        1    Xinil        47           Akira       5           2
4        1    Xinil       591  Amaenaide yo!!       6           2

   average_ratings
0         8.333333
1         7.000000
2         7.250000
3         7.500000
4         5.500000
```

```
dict_title_to_id = dict(zip(score_dataset_with_avg_ratings["Anime
Title"], score_dataset_with_avg_ratings["anime_id"] ))
dict_id_to_title = {v : k for k, v in dict_title_to_id.items()}

# Créer le DataFrame df_0 avec les caractéristiques de chaque anime
df_0 = pd.DataFrame({
    'anime_id': nb_ratings_per_anime.index,  # Identifiant de l'anime
    'anime_title': nb_ratings_per_anime.index.map(dict_id_to_title),
# Titre de l'anime
    'average_ratings': average_ratings_per_anime,  # Note moyenne
    'nb_ratings': nb_ratings_per_anime  # Nombre de notes
})

# Supprimer les animes avec des valeurs manquantes (si nécessaire)
df_0 = df_0.dropna()

# Afficher les premières lignes de df_0 pour vérifier
print(df_0.head())
```

```
          anime_id                      anime_title
average_ratings  \
anime_id

1                1                     Cowboy Bebop         8.833333

5                5  Cowboy Bebop: Tengoku no Tobira         7.833333

6                6                           Trigun         8.500000

7                7                Witch Hunter Robin         8.000000

15              15                     Eyeshield 21         7.000000


          nb_ratings
anime_id
1                  6
```

```
5                    6
6                    4
7                    3
15                   2
```

```
from sklearn.neighbors import NearestNeighbors

def recommander_animes_utilisateur(user_id, score_dataset, df_0,
model, n_recommendations=10):
    # Filtrer le dataset pour obtenir les animes notés par
l'utilisateur
    animes_utilisateur = score_dataset[score_dataset['user_id'] ==
user_id]

    # Si l'utilisateur n'a pas d'animes, retourner un message vide
    if animes_utilisateur.empty:
        print(f"Aucun anime trouvé pour l'utilisateur {user_id}")
        return []

    # Choisir un anime de l'utilisateur pour la recommandation (ici,
le premier)
    anime_id = animes_utilisateur.iloc[0]['anime_id']

    # Extraire le vecteur des caractéristiques de l'anime choisi par
l'utilisateur
    film_1_vector = df_0[df_0['anime_id'] == anime_id]
[['average_ratings', 'nb_ratings']]

    # Utiliser le modèle KNN pour trouver les voisins les plus proches
(recommandations)
    distance, neig = model.kneighbors(film_1_vector.to_numpy(),
n_neighbors=n_recommendations + 1)  # +1 pour ignorer l'anime lui-même

    print("Anime Name : ")
    print(df_0[df_0['anime_id'] == anime_id])

    # Créer une liste pour stocker les recommandations
    recommendations = []

    # Parcourir les voisins et extraire les titres d'animes
recommandés
    for i in range(1, len(distance[0])):  # Commencer à 1 pour ignorer
l'anime de départ
        titre = df_0.iloc[neig[0][i]]['anime_title']
        avg_rating = df_0.iloc[neig[0][i]]['average_ratings']
        nb_ratings = df_0.iloc[neig[0][i]]['nb_ratings']

        # Ajouter la recommandation à la liste
        recommendations.append((titre, avg_rating, nb_ratings))
        print(f"{titre} recommandé avec une note moyenne de
```

```
{avg_rating:.2f} et {nb_ratings} notes.")

    return recommendations

# Sélectionner les colonnes de caractéristiques (note moyenne et
nombre de notes)
features = df_0[['average_ratings', 'nb_ratings']]

# Créer et entraîner le modèle KNN
model_knn = NearestNeighbors(n_neighbors=11, algorithm='auto')
model_knn.fit(features)

# Exemple d'appel de la fonction de recommandation
topN_recommendations = recommander_animes_utilisateur(user_id=1,
score_dataset=score_dataset_with_avg_ratings, df_0=df_0,
model=model_knn, n_recommendations=10)
```

```
Anime Name :
          anime_id anime_title  average_ratings  nb_ratings
anime_id
21              21   One Piece         8.333333           3
Koukaku Kidoutai: Stand Alone Complex recommandé avec une note moyenne
de 8.33 et 3 notes.
Sword Art Online recommandé avec une note moyenne de 8.33 et 3 notes.
Hajime no Ippo: New Challenger recommandé avec une note moyenne de
8.33 et 3 notes.
Mushishi recommandé avec une note moyenne de 8.33 et 3 notes.
One Piece recommandé avec une note moyenne de 8.33 et 3 notes.
One Punch Man recommandé avec une note moyenne de 8.33 et 3 notes.
Tengen Toppa Gurren Lagann recommandé avec une note moyenne de 8.67 et
3 notes.
Higurashi no Naku Koro ni recommandé avec une note moyenne de 8.67 et
3 notes.
Shingetsutan Tsukihime recommandé avec une note moyenne de 8.67 et 3
notes.
Fruits Basket recommandé avec une note moyenne de 8.67 et 3 notes.
```

```
C:\Users\anasa\AppData\Roaming\jupyterlab-desktop\jlab_server\Lib\
site-packages\sklearn\base.py:493: UserWarning: X does not have valid
feature names, but NearestNeighbors was fitted with feature names
  warnings.warn(
```